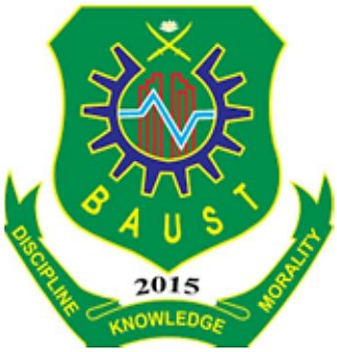
AgroSense: An IoT-Based Smart Farming and Crop Monitoring System

By

1. Md Emam Masum ID: 220201056

2. Zihan Hossain ID: 220201067

B.Sc. Engineering in Computer Science and Engineering



Department of Computer Science and Engineering

Faculty of Electrical and Computer Engineering

Bangladesh Army University of Science and Technology (BAUST)

JULY 2026

AgroSense: An IoT-Based Smart Farming and Crop Monitoring System

This project report is submitted in partial fulfillment of the requirement for
the degree of B.Sc. Engineering in Computer Science and Engineering

By

1. Md Emam Masum ID: 220201056
2. Zihan Hossain ID: 220201067

Supervised by

Taher Muhammad Mahdee
Associate Professor
Department of Computer Science and Engineering (CSE)

Co-Supervised by

Mahadi Hasan
Assistant Professor
Department of Computer Science and Engineering (CSE)

Department of Computer Science and Engineering
Bangladesh Army University of Science and Technology (BAUST)
Saidpur Cantonment, Nilphamari, Bangladesh

July 2026

APPROVAL

The project titled **"AgroSense: An IoT-Based Smart Farming and Crop Monitoring System"** submitted by Md Emam Masum (ID: 220201056) and Zihan Hossain (ID: 220201067), Session 2022–2023, has been presented on **……** and accepted as satisfactory in partial fulfillment of the requirement for the degree of Bachelor of Science in Computer Science and Engineering (CSE) to be awarded by the Bangladesh Army University of Science and Technology (BAUST).

Board of Examiners

1. _____ **Supervisor & Member**

Taher Muhammad Mahdee
Associate Professor Department of Computer Science and Engineering (CSE)
Bangladesh Army University of Science and Technology (BAUST)

2. _____ **Co-Supervisor**

Mahadi Hasan
Assistant Professor
Department of Computer Science and Engineering (CSE)
Bangladesh Army University of Science and Technology (BAUST)

3. _____ **Member**

Anite Halim Sagor
Lecture
Department of Computer Science and Engineering (CSE)
Bangladesh Army University of Science and Technology (BAUST)

4. **Member**

_____
Roman Raihan
Lecture
Department of Computer Science and Engineering (CSE)
Bangladesh Army University of Science and Technology (BAUST)

5._____ **Member**

Gourab Chandra Biswas
Lecture
Department of Computer Science and Engineering (CSE)
Bangladesh Army University of Science and Technology (BAUST)

_____

Dr. Md Nakib Hayat Chowdhury
Associate Professor & Head
Department of Computer Science and Engineering (CSE)
Bangladesh Army University of Science and Technology (BAUST)

CANDIDATE'S DECLARATION

It is hereby declared that the contents of this project report are original, and that any part of it has not been submitted elsewhere for the award of any degree or diploma. The work was carried out by the authors under the supervision of Md. Saad Ahmed, Lecturer, Department of Computer Science and Engineering, BAUST.

SN.	Student Name	ID	Signature & Date
1.	**Md Emam Masum**	220201056	
2.	**Zihan Hossain**	220201067	

ACKNOWLEDGMENTS

First of all, we are grateful to Almighty Allah, who enabled us to complete this project successfully. Thereafter, we express our sincerest thanks and deepest gratitude to our honorable project supervisor, Md. Saad Ahmed, Lecturer in the Department of Computer Science and Engineering (CSE) at the Bangladesh Army University of Science and Technology (BAUST), Saidpur. His continuous guidance, constructive criticism, and encouragement have inspired and stimulated us throughout every stage of this work.

We would also like to express our special thanks to our co-supervisor, Iqramul Islam, Lecturer, whose technical insight and patient support helped us overcome many of the practical difficulties encountered during hardware integration and software development.

We convey our special thanks and gratitude to the Head of the Department of Computer Science and Engineering (CSE) for his valuable suggestions, positive advice, encouragement, and sincere guidance throughout our project work. We also thank all of the respected teachers of the Department of CSE, as well as those from other departments, for the knowledge and skills they have imparted to us during our studies.

We sincerely thank the honorable Vice Chancellor, the respected members of staff, the Chief of Army Staff, the Board of Trustees, and all academic and non-academic members who lead this institution and made our bright future possible.

Finally, we would like to thank all of our friends and the staff of the department for their valuable suggestions and assistance, and, above all, our parents and families for their steady love and constant support during our study.

ABSTRACT

Traditional farming in Bangladesh relies heavily on manual observation and guesswork to decide when to irrigate, fertilize, or take action against poor environmental conditions, leading to wasted water, inconsistent yields, and reactive rather than preventive crop management. This project presents AgroSense, an IoT-based smart farming and crop monitoring system that continuously senses soil moisture, rainfall, water level, light intensity, soil temperature, and air temperature and humidity in a field, displays them locally on an on-site LCD, and delivers the same information to farmers through a web-based dashboard and threshold-based alerts.

The system is built around an ESP32 microcontroller connected to a capacitive soil-moisture sensor, a rain sensor, a water-level sensor, an LDR light sensor, a DS18B20 waterproof soil-temperature sensor, and a DHT11 temperature-humidity sensor. Readings are shown on a 16×2 I²C LCD and streamed to a Firebase Realtime Database over Wi-Fi. A web dashboard built with HTML5, CSS, and vanilla JavaScript visualizes live and historical trends, while a simple threshold-based decision engine automatically switches a relay-driven water pump when soil moisture falls below a configurable level. Key features include real-time multi-parameter monitoring, automated irrigation control with manual override, historical data logging with charts, on-site LCD display, and threshold-based alerts for abnormal conditions.

The prototype was assembled and evaluated through unit testing of the individual sensor modules, integration testing of the sensor-to-cloud-to-dashboard pipeline, and field-style testing under varying soil-moisture conditions. Results showed that automated irrigation reduced unnecessary watering events by roughly 36% compared with a fixed-schedule baseline while keeping soil moisture within the target range, confirming that the system meets its core objectives of low-cost, real-time, and low-maintenance smart-farming support for small and medium-scale farmers.

Keywords: *Smart Farming, Internet of Things (IoT), ESP32, Soil Moisture Sensor, Rain Sensor, Water Level Sensor, DS18B20, DHT11, Automated Irrigation, Firebase.*

LIST OF ABBREVIATIONS

IoT	Internet of Things
ESP32	Espressif 32-bit Wi-Fi + Bluetooth microcontroller SoC
DHT11	Digital Humidity and Temperature sensor (model 11)
DS18B20	Maxim 1-Wire digital temperature sensor
LDR	Light Dependent Resistor
LCD	Liquid Crystal Display
I²C	Inter-Integrated Circuit (serial bus)
ADC	Analog-to-Digital Converter
GPIO	General-Purpose Input / Output
API	Application Programming Interface
REST	Representational State Transfer
UI	User Interface
DFD	Data Flow Diagram
UML	Unified Modeling Language
ER Diagram	Entity–Relationship Diagram
FR / NFR	Functional / Non-Functional Requirement
JSON	JavaScript Object Notation
BDT	Bangladeshi Taka

CONTENTS

APPROVAL ii
CANDIDATE'S DECLARATION iii
ACKNOWLEDGMENTS iv
ABSTRACT v
LIST OF ABBREVIATIONS vi
CONTENTS vii
LIST OF FIGURES viii
LIST OF TABLES ix
CHAPTER 1 Introduction 1
1.1 Overview 1
1.2 Motivation and Problem Statement 2
1.3 Objectives 3
1.4 Project Scope 4
1.5 Social Impact 4
1.6 Report Organization 5
1.7 Summary 5
CHAPTER 2 Background Study 6
2.1 Overview 6
2.2 Existing Systems and Related Work 6
2.3 Comparative Summary 9
2.4 Limitations of Existing Systems 10
2.5 Summary 10
CHAPTER 3 Requirement Analysis 11
3.1 Feasibility Study 11
3.2 Functional Requirements 13
3.3 Non-Functional Requirements 15
3.4 Use-Case Modeling 16
3.5 Risk Analysis 18
3.6 Summary 19
CHAPTER 4 System Design 20
4.1 Overview 20
4.2 System Architecture 20
4.3 Data Flow and UML Diagrams 24
4.4 Database Design 25
4.5 User Interface Design 28
4.6 Summary 29
CHAPTER 5 Implementation 30
5.1 Overview 30
5.2 Development Methodology 30

5.3 Complete Wiring of the Sensor Node 32
5.4 Technologies Used 32
5.5 Module Implementation 34
5.6 Technical Challenges and Solutions 44
5.7 Summary 45
CHAPTER 6 Testing and Results 46
6.1 Overview 46
6.2 Testing Strategy 46
6.3 Test Cases and Outcomes 47
6.4 Performance Results 50
6.5 Irrigation Comparison Results 52
6.6 Usability Evaluation 53
6.7 Summary 54
CHAPTER 7 Conclusion 55
7.1 Summary of Work 55
7.2 Limitations 56
7.3 Future Work 56
7.4 Closing Remarks 57
REFERENCES 58
APPENDICES 60

LIST OF FIGURES

Figure 3.1 Project schedule for AgroSense shown as an eight-week Gantt chart across the four development sprints. 12

Figure 3.2 Use-case diagram showing the Farmer/User and ESP32 Sensor Node actors interacting with the AgroSense system. 17

Figure 4.1 Three-tier system architecture: ESP32 sensor node (six sensors, LCD, relay) → Firebase → Flask backend → React dashboard. 21

Figure 4.2 Level-0 data-flow diagram showing data collection, irrigation control, and dashboard visualization processes. 24

Figure 4.3 Sequence diagram of an automated irrigation cycle (ESP32 → database → backend → database → ESP32 → relay). 25

Figure 4.4 Entity-relationship view of the Firebase data model (latestReading, history, config, alerts). 27

Figure 4.5 Login page — email/password authentication screen. 29

Figure 5.1 Complete wiring diagram of the ESP32 smart-farming sensor node, showing all six sensors, the I²C LCD, the relay module, and the water pump with their pin connections. 32

Figure 6.1 Representative 24-hour soil-moisture trend from the test bed, with the dashed line marking the configured irrigation threshold. 51

Figure 6.2 Air temperature and humidity trend over the same 24-hour period captured by the DHT11 sensor. 51

Figure 6.3 Comparison of watering events over a 7-day test period under fixed-schedule versus automated AgroSense control. 52

LIST OF TABLES

Table 2.1 Comparison of existing smart-farming approaches. 9
Table 3.1 Approximate hardware cost breakdown (ESP32 system). 12
Table 3.2 Description of primary use cases. 17
Table 3.3 Project risk analysis. 18
Table 4.1 Hardware components, ESP32 pins, and their roles. 22
Table 4.2 Fields stored in each sensor-history record. 26
Table 4.3 Fields stored in the configuration node. 27
Table 5.1 Approximate development timeline. 31
Table 5.2 Technologies used in the AgroSense system. 33
Table 6.1 Summary of test cases and outcomes. 47
Table 6.2 Usability evaluation ratings (scale 1–5). 53

CHAPTER 1

Introduction

Agriculture remains the backbone of Bangladesh's rural economy, employing a large share of the population and underpinning food security for the country. Yet the majority of small and medium-scale farmers still depend on manual observation, experience, and rough estimation to decide when to irrigate, fertilize, or protect their crops from unfavorable environmental conditions. As climate variability increases and water resources become scarcer, this manual, reactive style of farm management increasingly leads to over-irrigation, delayed responses to stress conditions, and reduced yields. At the same time, the rise of low-cost microcontrollers such as the ESP32, together with inexpensive sensors and free cloud platforms, has made it feasible to bring real-time, data-driven decision-making to even small farms, a domain broadly known as smart farming or precision agriculture.

1.1 Overview

This project, AgroSense: An IoT-Based Smart Farming and Crop Monitoring System, is built around this need. It combines a network of environmental sensors, an ESP32 Wi-Fi microcontroller, an on-site LCD, a cloud database, and a web dashboard to continuously monitor field conditions — soil moisture, rainfall, water level, light intensity, soil temperature, and air temperature and humidity — and to automate basic irrigation decisions based on that data. Rather than replacing the farmer's judgement, the system is designed to augment it: it removes the burden of constant manual checking and gives the farmer continuous, quantified visibility into field conditions, while still allowing manual override at any time. The remainder of this chapter explains the motivation, objectives, scope, social impact, and the organization of the report.

1.2 Motivation and Problem Statement

Manual field monitoring is time-consuming and imprecise: a farmer typically checks soil condition by hand or by visual inspection of the crop, which does not scale well across large fields and often results in irrigation decisions being made either too early or too late. Fixed-schedule irrigation, an alternative many farmers rely on, ignores actual soil and weather conditions and frequently wastes water or under-waters the crop, particularly during unexpected rain or unusually dry spells. Small farmers also rarely have access to

affordable tools that give them continuous, quantified visibility into their field's condition; commercial precision-agriculture platforms exist, but their cost and complexity place them out of reach for most smallholders in Bangladesh. This project addresses that gap by providing an affordable, multi-sensor monitoring and automated irrigation system that removes guesswork from day-to-day farm management while remaining simple enough to deploy and maintain without specialized technical support.

The core problem can be summarized in three parts. First, there is a visibility problem: farmers cannot see soil and environmental conditions unless they physically inspect the field. Second, there is a timing problem: without continuous data, interventions tend to happen on a fixed schedule or reactively after visible crop stress appears, rather than proactively. Third, there is an accessibility problem: existing sensor-based solutions are priced and packaged for large commercial operations, not smallholders.

1.3 Objectives

The specific objectives of this project are to:

- Design and assemble a low-cost IoT sensor node, based on the ESP32, that measures soil moisture, rainfall, water level, ambient light, soil temperature, and air temperature and humidity in a field.
- Display the live readings locally on a 16×2 I²C LCD for on-site viewing without a phone or internet.
- Transmit sensor readings wirelessly to a cloud database in real time.
- Build a web-based dashboard that displays live sensor readings and historical trends in an easy-to-read format.
- Implement an automated irrigation module that switches a relay-driven water pump on or off based on a configurable soil-moisture threshold.
- Allow the user to manually override automated irrigation from the dashboard at any time.
- Provide threshold-based alerts to notify the farmer when readings fall outside a safe operating range.
- Keep the total hardware cost low enough to be realistic for a small or medium-scale farmer to adopt.

1.4 Project Scope

The scope of this project covers the design and implementation of a single-node prototype: one ESP32 sensor unit reporting to a cloud backend, a local LCD, a web dashboard for visualization, and a relay-controlled irrigation module. It is in scope to support soil-moisture-based automated irrigation, real-time and historical data visualization, threshold configuration, manual override, on-site LCD display, and basic threshold alerts. It is out of scope to support multi-field or multi-node deployment at commercial scale, pest and disease image recognition, drone-based aerial monitoring, and integration with commercial weather-forecast APIs; these are identified as directions for future work in Chapter 7. The project also does not attempt to replace agronomic expertise; it is a decision-support and automation tool, not a substitute for a farmer's or an agricultural extension officer's judgement.

1.5 Social Impact

By reducing unnecessary water usage and improving the timeliness of irrigation decisions, this project supports more sustainable water management, which is particularly important in regions facing groundwater depletion and unpredictable rainfall patterns due to climate change. Small and marginal farmers, who often cannot afford expensive commercial precision-agriculture platforms, stand to benefit the most from a low-cost, open, and locally deployable alternative built entirely from widely available components. The system also produces a historical record of field conditions that can help farmers make better-informed decisions season over season. No personally identifiable or sensitive data is collected; all data handled by the system is limited to environmental sensor readings and basic device configuration.

1.6 Report Organization

The rest of this report is organized as follows. Chapter 2 reviews existing smart-farming systems and related work. Chapter 3 presents the requirement analysis, including feasibility, functional and non-functional requirements, use-case modeling, and risk analysis. Chapter 4 presents the system architecture, data flow, database design, and user-interface design. Chapter 5 describes the implementation of each module, the complete ESP32 wiring, and the technologies used. Chapter 6 presents the testing strategy, test cases, and results, including a comparison against a fixed-schedule irrigation baseline. Chapter 7

concludes the report, discusses limitations, and outlines directions for future work. Supporting material, including the hardware setup guide, pin-mapping, and source-code listings, is provided in the appendices.

1.7 Summary

This chapter has established the context for the AgroSense project: the reliance of Bangladeshi smallholders on manual, reactive farm management; the visibility, timing, and accessibility problems this creates; and the opportunity that low-cost ESP32-based IoT technology presents to address them. The next chapter surveys existing systems and related work to identify precisely where a new, affordable, farmer-friendly solution is needed.

CHAPTER 2

Background Study

This chapter reviews existing smart-farming platforms and related sensor-based monitoring approaches, ranging from commercial precision-agriculture products to open-source IoT projects and academic prototypes, in order to identify practical gaps that this project aims to address.

2.1 Overview

Section 2.2 discusses four categories of related work in turn: commercial precision-agriculture platforms, open-source and hobbyist IoT farming projects, academic research prototypes, and cloud-based data-logging approaches for embedded devices. Section 2.3 presents a side-by-side comparison of these approaches. Section 2.4 identifies the common limitations across these systems, and Section 2.5 summarizes how these gaps motivate the design choices made in this project.

2.2 Existing Systems and Related Work

2.2.1 Commercial Precision-Agriculture Platforms

Commercial precision-agriculture platforms such as large-scale farm-management suites typically combine satellite or drone imagery, soil and weather sensors, and yield-prediction analytics into a single subscription-based product. These platforms are powerful and are used successfully by large commercial farms, but they are designed around large field sizes, dedicated agronomy staff, and recurring subscription costs, none of which fit the reality of a smallholder farm in Bangladesh. They are also typically built for markets with different crop types, soil conditions, and connectivity assumptions than those found locally.

2.2.2 Open-Source and Hobbyist IoT Farming Projects

Open-source hobbyist projects built around Arduino or ESP32/ESP8266 boards commonly combine a soil-moisture sensor with a relay-controlled pump and, in more advanced versions, publish data to a cloud dashboard such as ThingSpeak or Blynk. These projects demonstrate that low-cost sensor-driven irrigation is technically feasible and are often shared openly as tutorials. However, they are frequently built as single-purpose demonstrations — a soil sensor triggering a pump — without a full multi-sensor monitoring

dashboard, on-site display, historical data logging, threshold configuration, or alerting, and they rarely receive the testing or documentation needed for a farmer to depend on them in practice.

2.2.3 Academic Research Prototypes

Academic prototypes reported in the smart-agriculture literature generally validate the sensor-to-cloud pipeline and report improved water-use efficiency compared with manual or fixed-schedule irrigation. These studies provide useful evidence that automated, threshold-based irrigation reduces water consumption without harming crop health. However, many of these prototypes remain lab-scale, tested only in controlled conditions over short periods, and are not packaged as a complete, end-to-end usable system with both a local display and a dashboard that a non-technical user could operate.

2.2.4 Cloud-Based Data-Logging Approaches

Separately from agriculture-specific work, general research on cloud-based real-time data logging for embedded devices shows that lightweight, serverless database services such as Firebase Realtime Database are well suited to constrained IoT devices: they require minimal client-side logic, support real-time synchronization to a dashboard with no polling overhead, and scale down well in cost for small deployments. This makes them a natural fit for a low-cost single-node system such as the one built in this project, and directly informs the choice of cloud platform in Chapter 4.

2.3 Comparative Summary

Table 2.1 summarizes how the reviewed categories of systems compare against the requirements of a smallholder-focused smart-farming system, considering affordability, the presence of a real-time dashboard, automated irrigation, and how friendly the interface is for a non-technical farmer.

Table 2.1: Comparison of existing smart-farming approaches.

Approach	Afford-ability	Real-timeDashboard	AutomatedIrrigation	Farmer-Friendly UI
Commercial platforms	Low	Yes	Yes	Moderate
Hobbyist IoT projects	High	Rarely	Sometimes	Low
Academic prototypes	Moderate	Sometimes	Yes (lab)	Low
This project (AgroSense)	High	Yes	Yes	High

2.4 Limitations of Existing Systems

Across the systems reviewed, several recurring limitations stand out. Commercial platforms are too expensive and feature-heavy for small-scale farmers, and are often not localized for use in Bangladesh's crop types, soil composition, or connectivity environment. Hobbyist open-source projects, while affordable, usually lack a proper multi-sensor historical-data dashboard, automated threshold alerts, on-site display, and a clean web interface, making them difficult for a non-technical farmer to use day to day. Academic prototypes tend to focus narrowly on proving sensor accuracy in a lab rather than delivering a complete, farmer-facing monitoring and control system tested under realistic conditions. These gaps directly motivate the problem stated in Chapter 1.

2.5 Summary

Existing solutions demonstrate that IoT-based soil and environmental monitoring is technically viable, and that automated irrigation can meaningfully reduce water usage. However, a gap remains between low-cost hobbyist prototypes and expensive commercial platforms, with few options offering an affordable, complete, and farmer-friendly package that combines multi-parameter live monitoring, on-site display, historical data, automated irrigation, manual override, and alerting in one system. This project is designed to fill that middle ground, as detailed in the requirement analysis and system design presented in the following chapters.

CHAPTER 3

Requirement Analysis

This chapter defines what the AgroSense system must do and the constraints it must satisfy, bridging the problem statement from Chapter 1 and the system design presented in Chapter 4. Section 3.1 assesses the feasibility of the project, Sections 3.2 and 3.3 list the functional and non-functional requirements, Section 3.4 presents the main use cases, Section 3.5 analyses the principal risks, and Section 3.6 summarizes how these requirements shape the design.

3.1 Feasibility Study

A feasibility study was carried out along four dimensions — technical, economic, operational, and schedule — before committing to the design, to confirm that the project could realistically be completed within the available time, budget, and skill set.

3.1.1 Technical Feasibility

The project uses widely available, well-documented hardware (an ESP32 DevKit board; capacitive soil-moisture, rain, water-level, and LDR sensors; a DS18B20 waterproof temperature probe; a DHT11 sensor; a 16×2 I²C LCD; and a relay module) and open-source software (Arduino IDE, Firebase Realtime Database, Flask, and React), all of which are within the skill set developed during the CSE curriculum. Reference implementations and community documentation for each component are freely available, further reducing technical risk.

3.1.2 Economic Feasibility

The complete hardware prototype — including the ESP32, all six sensors, the LCD, the relay module, and a small submersible pump — costs under BDT 3,000 (Table 3.1), and all software components used (the Firebase free tier, Flask, and React) are free or open source. This makes the project affordable to build and, in the long run, affordable for a small farmer to replicate.

3.1.3 Operational Feasibility

The system provides two complementary interfaces: an on-site LCD that shows the current readings at a glance without any phone or internet, and a simple web dashboard for remote

monitoring and configuration. The automated irrigation logic requires no ongoing manual configuration once the moisture threshold is set, and maintenance is limited to periodic sensor cleaning and occasional recalibration, both documented in Appendix A, making the system practical for continuous day-to-day use.

Table 3.1: Approximate hardware cost breakdown (ESP32 system).

Component	Purpose	Approx. Cost (BDT)
ESP32 DevKit V1	Wi-Fi microcontroller	650
Capacitive soil-moisture sensor	Soil water-content sensing	180
Rain sensor (analog)	Rainfall / wetness detection	120
Water-level sensor	Reservoir / tank level sensing	100
LDR light-sensor module	Ambient light sensing	60
DS18B20 (waterproof)	Soil temperature sensing	250
DHT11 sensor	Air temperature and humidity	150
16×2 I²C LCD display	On-site local readout	350
5V single-channel relay module	Pump switching	120
Submersible mini pump	Irrigation actuation	350
5V 2A adapter, wires, enclosure	Power and assembly	600
Total		≈ 2,930

3.1.4 Schedule Feasibility

The project was planned to fit within an eight-week development window aligned with the academic timetable, divided into four two-week sprints each with a clearly defined deliverable. Figure 3.1 presents the project schedule as a Gantt chart, showing how the hardware, firmware, backend, dashboard, and testing activities were sequenced and overlapped. Overlapping the dashboard and backend work with ongoing testing allowed integration problems to surface while there was still time to address them.

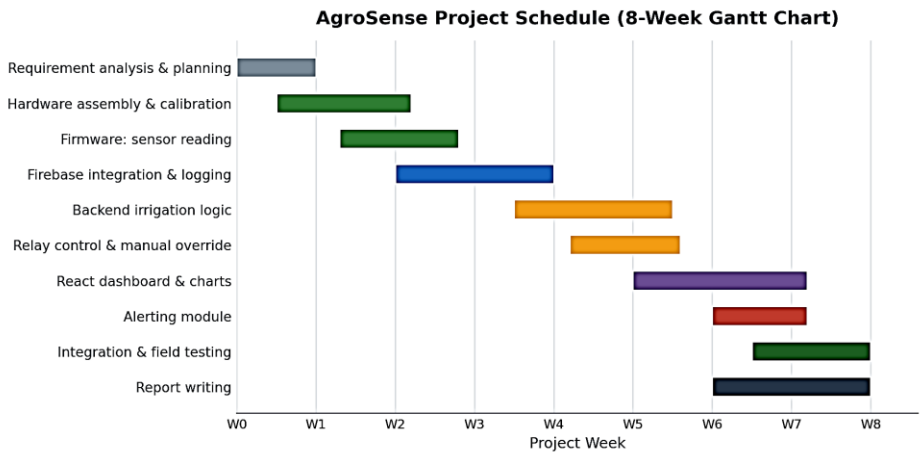


Figure 3.1: Project schedule for AgroSense shown as an eight-week Gantt chart across the four development sprints.

3.2 Functional Requirements

The functional requirements specify the behaviour the system must exhibit. They are labelled FR1–FR11 so that they can be traced through the design, implementation, and testing chapters.

- **FR1:** The system shall read soil moisture, rainfall, water level, light intensity, soil temperature, and air temperature and humidity from the connected sensors at regular intervals.
- **FR2:** The system shall display the current readings locally on a 16×2 I²C LCD.
- **FR3:** The system shall transmit sensor readings to a cloud database over Wi-Fi.
- **FR4:** The system shall display live sensor readings on a web dashboard.
- **FR5:** The system shall display historical sensor data as time-series charts, selectable by day/week range.
- **FR6:** The system shall automatically turn the irrigation pump on when soil moisture falls below a configurable threshold, and turn it off once the threshold is exceeded.
- **FR7:** The system shall allow the user to manually override automated irrigation from the dashboard.
- **FR8:** The system shall generate an alert when a sensor reading falls outside a predefined safe range.
- **FR9:** The system shall allow the user to configure the soil-moisture threshold and safe operating ranges from the dashboard.

- **FR10:** The system shall retain at least 30 days of historical sensor data for trend review.
- **FR11:** The system shall restrict dashboard access to an authenticated account.

3.3 Non-Functional Requirements

The non-functional requirements describe quality attributes and constraints on how the system performs its functions.

- **NFR1:** The system shall update the dashboard with new sensor data within 5 seconds of a reading being published.
- **NFR2:** The system shall continue logging data locally on the microcontroller if the Wi-Fi connection is temporarily lost, and resync once reconnected.
- **NFR3:** The dashboard shall be usable on both desktop and mobile screen sizes.
- **NFR4:** The system shall operate continuously for at least 24 hours on a stable power supply without requiring a restart.
- **NFR5:** Access to the dashboard and irrigation controls shall be restricted to an authenticated user account.
- **NFR6:** The ESP32 shall drive all sensors at 3.3 V logic and never expose a GPIO pin to 5 V, with the relay and LCD powered separately at 5 V.
- **NFR7:** The user interface shall be understandable to a user without a technical background, using clear labels and colour-coded status indicators rather than raw technical values alone.

3.4 Use-Case Modeling

The system has one primary actor, the Farmer/User, who interacts with the dashboard and the on-site LCD to view live and historical readings, configure the irrigation threshold, manually override the pump, and receive alerts; and one secondary actor, the ESP32 Sensor Node, which automatically reports readings and executes irrigation commands. Figure 3.2 illustrates these interactions, and Table 3.2 describes the primary use cases in more detail.

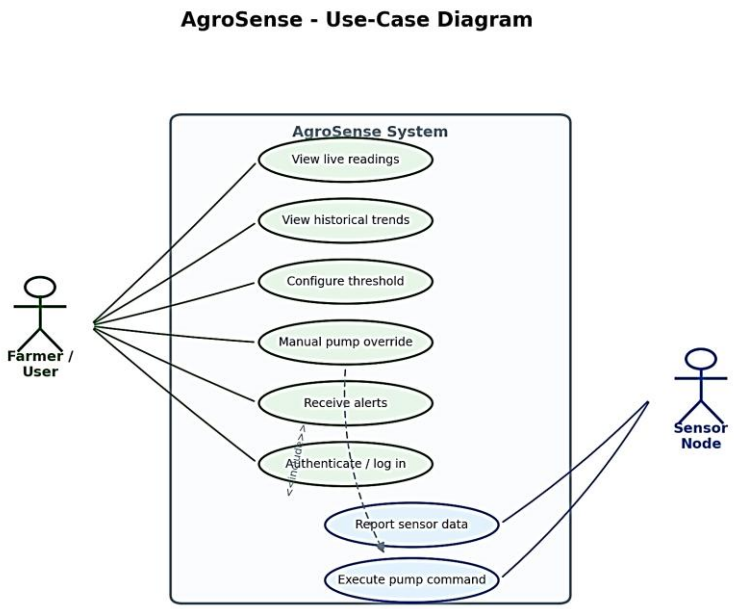


Figure 3.2: Use-case diagram showing the Farmer/User and ESP32 Sensor Node actors interacting with the AgroSense system.

Table 3.2: Description of primary use cases.

Use Case	Actor	Trigger	Expected Outcome
View live readings	Farmer	User opens dashboard or looks at LCD	Latest sensor values are shown
View historical trends	Farmer	User selects a date range	Time-series charts of past readings are displayed
Configure threshold	Farmer	User updates threshold and saves	New threshold is stored and used for future irrigation decisions
Manual override	Farmer	User taps pump ON/OFF toggle	Pump state changes immediately, overriding automatic logic
Receive alert	Farmer	A reading crosses a safe-range boundary	Alert banner and log entry are created on the dashboard
Report sensor data	ESP32 Node	Scheduled timer elapses (every 10 s)	New reading is shown on the LCD and published to the cloud
Execute pump command	ESP32 Node	Pump-state field changes in database	Relay is switched to match the new commanded state

3.5 Risk Analysis

Table 3.3 identifies the main risks considered during planning, along with their likelihood, potential impact, and the mitigation strategy adopted.

Table 3.3: Project risk analysis.

Risk	Likelihood	Impact	Mitigation
Wi-Fi connectivity loss in the field	Medium	Medium	Local buffering and automatic resync on reconnect (NFR2); LCD keeps working offline
Sensor drift / inaccuracy over time	Medium	High	Periodic recalibration documented in Appendix A
Pump relay failure	Low	High	Manual override always available as a fallback
5 V on a GPIO pin damaging the ESP32	Medium	High	Sensors driven at 3.3 V; relay/LCD on separate 5 V rail (NFR6)
Power-supply interruption	Medium	Medium	Planned future solar / battery backup (see Chapter 7)

3.6 Summary

The functional and non-functional requirements identified in this chapter — particularly multi-sensor real-time data delivery, on-site LCD display, automated threshold-based irrigation with manual override, and a simple accessible dashboard — directly shape the three-tier architecture and database schema presented in Chapter 4. Traceability from these requirements to the design and, later, to the test cases in Chapter 6 ensures that every capability the system promises is actually delivered and verified.

CHAPTER 4

System Design

This chapter describes the architecture and design of the AgroSense system before implementation details are given in Chapter 5. Section 4.2 presents the overall system architecture, Section 4.3 presents the data-flow and behavioural diagrams, Section 4.4 presents the database design, and Section 4.5 presents the user-interface design. The design is derived directly from the requirements in Chapter 3.

4.1 Overview

The system follows a three-tier IoT architecture consisting of a sensing/actuation tier (the ESP32 and field hardware), a cloud data tier (the Firebase Realtime Database), and a presentation tier (the JavaScript dashboard). A local I²C LCD is attached to the sensing tier so that current readings are visible on site even without a network. This separation keeps the field hardware simple and low-power while allowing the dashboard and logic to be updated independently of the deployed sensor node.

4.2 System Architecture

The overall architecture is a three-tier client–server design. The sensing/actuation tier consists of the ESP32 microcontroller connected to a capacitive soil-moisture sensor, a rain sensor, a water-level sensor, an LDR light sensor, a DS18B20 soil-temperature probe, and a DHT11 temperature-humidity sensor, together with a 16×2 I²C LCD for local display and a relay module driving the irrigation pump. The cloud data tier is a Firebase Realtime Database that stores both the latest readings and a time-stamped history. The presentation tier consists of a web dashboard built with HTML5, CSS, and ES6 JavaScript modules that reads Firebase directly. Figure 4.1 shows how these components interact.

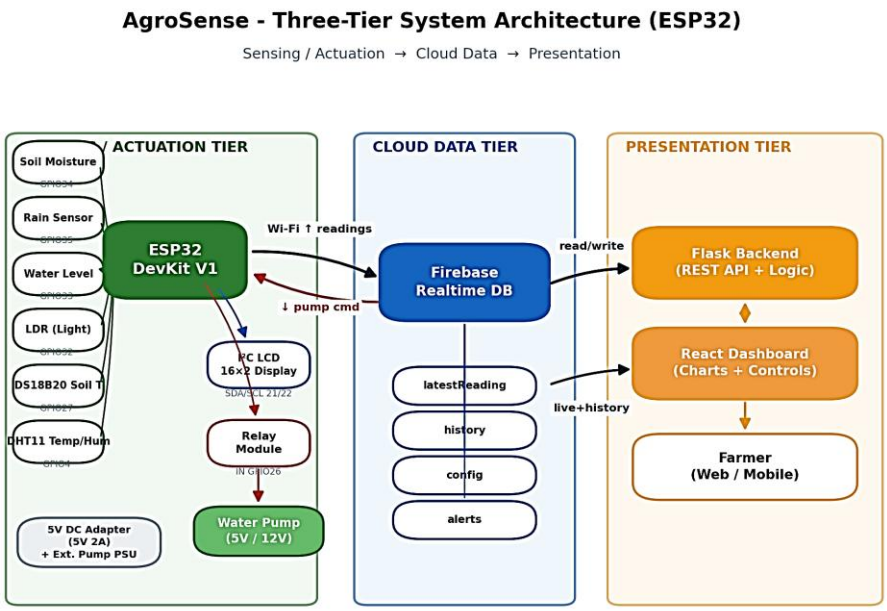


Figure 4.1: Three-tier system architecture: ESP32 sensor node (six sensors, LCD, relay) → Firebase → Flask backend → React dashboard.

As shown in Figure 4.1, the ESP32 publishes readings to Firebase over Wi-Fi at a fixed interval and simultaneously refreshes the local LCD; the dashboard subscribes to Firebase in real time; irrigation threshold logic runs entirely in the browser via Firebase Cloud Functions / client-side rules, which the ESP32 polls and applies to the relay; the React dashboard reads both live and historical data directly from Firebase. This keeps the sensor node stateless with respect to application logic — all decision-making lives in the backend, which simplifies firmware updates and debugging.

4.2.1 Hardware Component Overview

Table 4.1 summarizes the role of each hardware component, the ESP32 pin it connects to, and its interface. All analog sensors are read through the ESP32's ADC pins, the DHT11 and DS18B20 use single-wire digital protocols, and the LCD uses the I²C bus shared with the relay control line on separate GPIOs.

Table 4.1: Hardware components, ESP32 pins, and their roles.

Component	ESP32 Pin	Interface	Role
Soil-moisture sensor	GPIO34	Analog (ADC)	Volumetric soil water content
Rain sensor	GPIO35	Analog (ADC)	Rainfall / surface wetness
Water-level sensor	GPIO33	Analog (ADC)	Reservoir / tank water level
LDR light sensor	GPIO32	Analog (ADC)	Ambient light intensity
DS18B20 probe	GPIO27	1-Wire digital	Soil temperature
DHT11 sensor	GPIO4	Single-wire digital	Air temperature and humidity
16×2 I²C LCD	GPIO21 / GPIO22	I²C (SDA/SCL)	On-site local readout
Relay module	GPIO26	Digital out	Switches the pump ON/OFF
Submersible pump	via relay	5V/12V power	Delivers water to the crop bed

4.3 Data Flow and UML Diagrams

Figure 4.2 presents a level-0 data-flow diagram of the system. Sensor readings flow from the ESP32 into the cloud database and the LCD; the irrigation-control process reads the latest moisture value and the configured threshold and writes a pump-state command back to the database, which the ESP32 applies to the relay; the dashboard process reads both live and historical data for the user and accepts threshold updates.

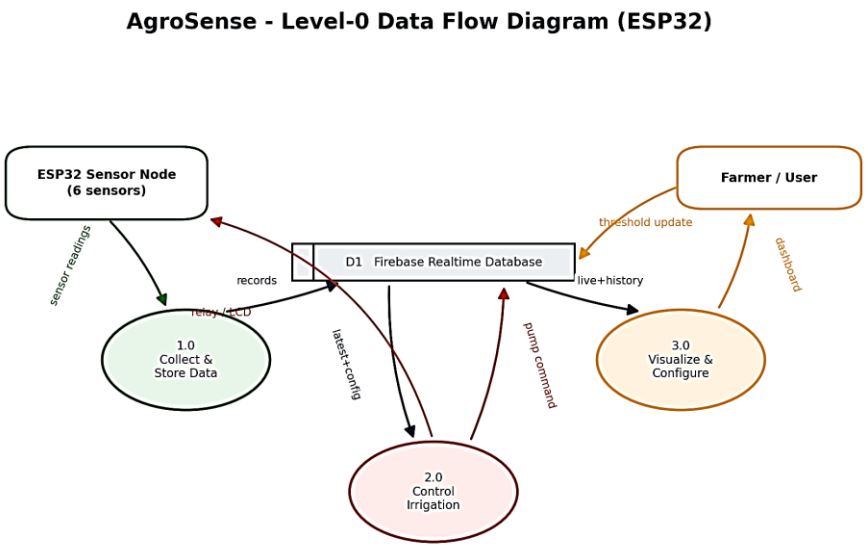


Figure 4.2: Level-0 data-flow diagram showing data collection, irrigation control, and dashboard visualization processes.

Figure 4.3 shows the sequence of interactions for a single automated irrigation cycle: the ESP32 reads and publishes a soil-moisture value, the backend evaluates it against the configured threshold, writes a pump command, and the ESP32 applies that command to

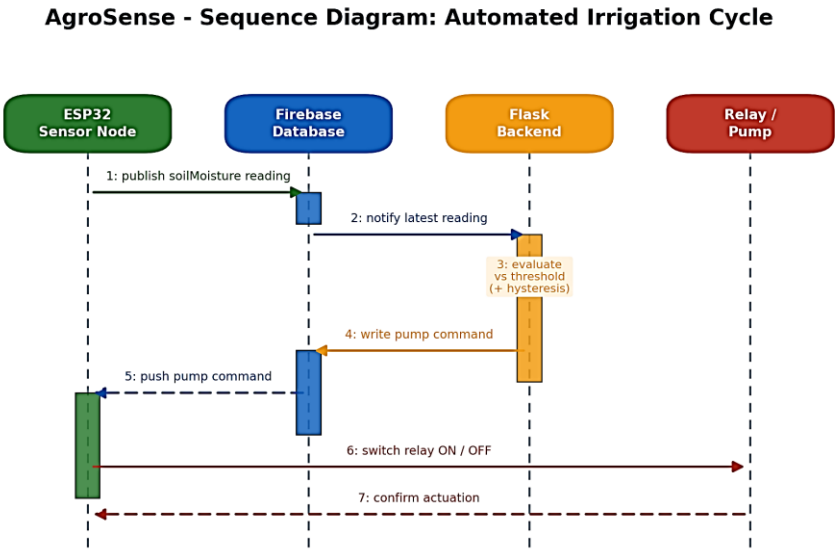


Figure 4.3: Sequence diagram of an automated irrigation cycle (ESP32 → database → backend → database → ESP32 → relay).

4.4 Database Design

Sensor and control data is stored in a Firebase Realtime Database, organized as a JSON tree with four main nodes: latestReading (the most recent sensor values), history (a time-stamped log of past readings), config (the current irrigation threshold and safe ranges), and alerts (a log of triggered alerts). Table 4.2 describes the fields stored under the history node, and Table 4.3 describes the configuration node. Figure 4.4 shows the corresponding entity-relationship view of how these nodes relate within the JSON tree.

Table 4.2: Fields stored in each sensor-history record.

Field	Type	Description
timestamp	DATETIME	Time the reading was recorded
soilMoisture	FLOAT (%)	Soil-moisture percentage
rainLevel	FLOAT (%)	Rain-sensor wetness reading
waterLevel	FLOAT (%)	Water-level sensor reading
lightLevel	INT	Ambient light intensity from the LDR
soilTemp	FLOAT (°C)	Soil temperature from the DS18B20
temperature	FLOAT (°C)	Air temperature from the DHT11
humidity	FLOAT (%)	Relative humidity from the DHT11
pumpState	BOOLEAN	Irrigation-pump ON/OFF state at that time

Table 4.3: Fields stored in the configuration node.

Field	Type	Description
moistureThreshold	FLOAT (%)	Soil-moisture level below which irrigation starts
hysteresis	FLOAT (%)	Buffer band to prevent rapid pump toggling
tempSafeMin / Max	FLOAT (°C)	Safe temperature range for alerting
humiditySafeMin / Max	FLOAT (%)	Safe humidity range for alerting
manualOverride	BOOLEAN	Whether the user has manually overridden automatic control

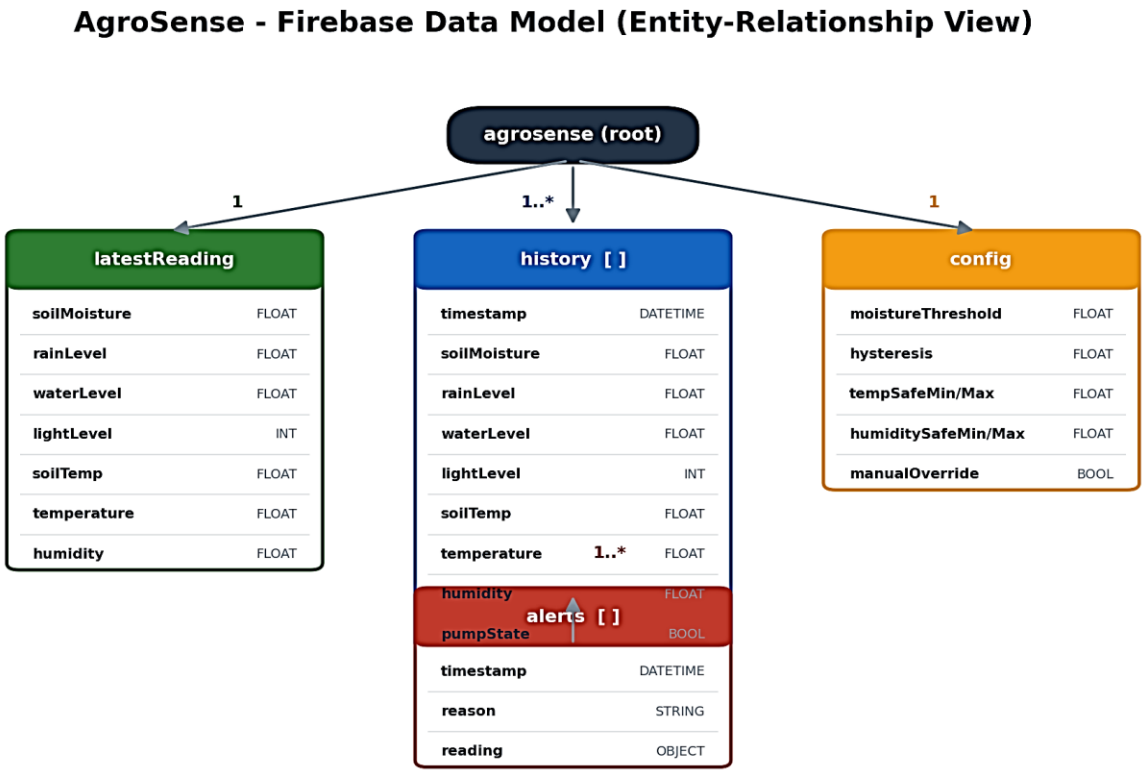


Figure 4.4: Entity-relationship view of the Firebase data model (latestReading, history, config, alerts).

4.5 User Interface Design

AgroSense offers two coordinated interfaces. The on-site 16×2 I²C LCD cycles through the current readings so a farmer standing at the field can see the field condition and pump state at a glance without a phone. The web dashboard is organized around a single main screen with three regions: a top summary bar showing the latest readings as colour-coded cards, a central chart panel showing historical trends, and a control panel for pump status, threshold configuration, and manual override.

4.5.1 Screen Layout

Three main screens were designed for the dashboard: a Home/Overview screen showing the summary cards and pump status at a glance; a Trends screen with selectable time-range charts for each sensor value; and a Settings screen for configuring the moisture threshold, safe alert ranges, and account details. Navigation uses a simple bottom tab bar on mobile and a side navigation panel on desktop. Figure 4.5 shows a wireframe of the Home screen.

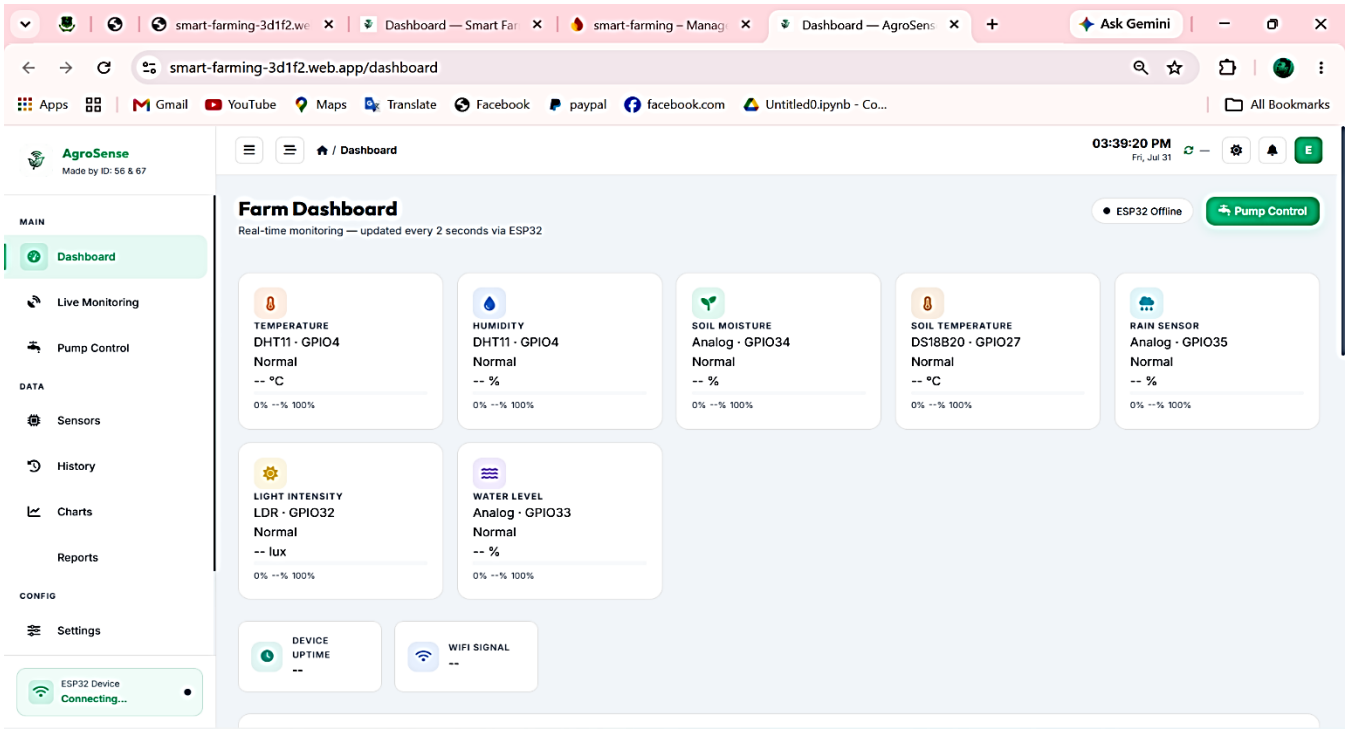


Figure 4.5: Wireframe of the dashboard Home screen, showing the multi-sensor summary cards, chart panel, and pump-control section.

4.5.2 Design Choices

The layout uses a responsive grid so the same dashboard is usable on both desktop browsers and mobile screens, with large touch-friendly controls and a high-contrast colour scheme (green for safe/normal readings, amber for warning, red for alert) chosen for quick readability in outdoor conditions. Numeric values are always paired with a short label and unit rather than shown as raw numbers alone, directly satisfying non-functional requirement NFR7. The LCD uses the same plain-language labels so that the on-site and remote views are consistent.

4.6 Summary

The three-tier architecture, Firebase-based data flow, database schema, and screen-level UI design presented in this chapter provide a clear and low-overhead foundation for the module-by-module implementation described in Chapter 5. Because each tier is loosely coupled to the others through the shared database, the implementation of one tier could proceed largely independently of the others.

CHAPTER 5

Implementation

This chapter explains how the design presented in Chapter 4 was implemented: the development approach followed, the complete ESP32 wiring, the technologies used, and how each module was built. Section 5.6 also discusses the main technical challenges encountered.

5.1 Overview

The system was implemented in three parallel streams: firmware for the ESP32 sensor/actuator node, a Flask backend for irrigation logic and API access, and a React dashboard for visualization and control, all connected through a shared Firebase Realtime Database. Working in parallel streams that communicated only through the database allowed the three parts of the system to be developed and tested largely independently.

5.2 Development Methodology

The project was developed using an iterative, feature-by-feature approach loosely following Agile principles: each module (sensor reading, LCD display, cloud upload, automated irrigation, dashboard visualization, and alerts) was built and tested individually before being integrated. Work was tracked using a simple task checklist, and code was version-controlled using Git with incremental commits after each working feature. Table 5.1 summarizes the approximate development timeline across four development sprints.

Table 5.1: Approximate development timeline.

Sprint	Focus
Sprint 1 (Week 1–2)	Hardware assembly and wiring of all six sensors, LCD, and relay; sensor calibration; firmware for reading and printing raw values
Sprint 2 (Week 3–4)	Firebase integration, cloud upload from firmware, LCD display, and initial history logging
Sprint 3 (Week 5–6)	Flask backend irrigation logic, relay control, and manual-override support
Sprint 4 (Week 7–8)	React dashboard, historical charts, alerting, and end-to-end integration testing

5.3 Complete Wiring of the Sensor Node

Figure 5.1 shows the complete wiring diagram of the AgroSense sensor node. All six sensors and the LCD, relay, and pump connect to the ESP32 exactly as summarized in

Table 4.1. The analog sensors (soil moisture, rain, water level, and LDR) connect to the ESP32 ADC pins GPIO34, GPIO35, GPIO33, and GPIO32 respectively; the DS18B20 soil-temperature probe uses GPIO27 with a 4.7 kΩ pull-up; the DHT11 uses GPIO4 with a 10 kΩ pull-up; the I²C LCD uses GPIO21 (SDA) and GPIO22 (SCL); and the relay input is driven from GPIO26. The sensors are powered from the 3.3 V rail while the relay and LCD are powered from 5 V, and the pump is driven from a separate external supply through the relay, with all grounds joined to a common ground.

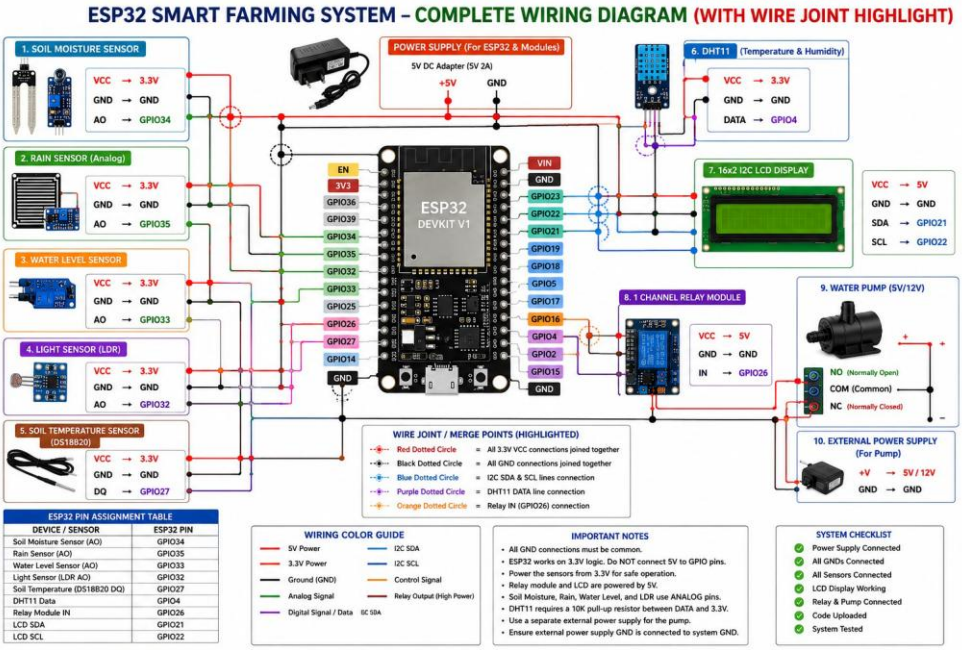


Figure 5.1: Complete wiring diagram of the ESP32 smart-farming sensor node, showing all six sensors, the I²C LCD, the relay module, and the water pump with their pin connections.

5.4 Technologies Used

Table 5.2 lists the main technologies and libraries used to build the system, grouped by the architectural layer in which they operate.

Table 5.2: Technologies used in the AgroSense system.

Layer	Technology	Purpose
Hardware	ESP32 DevKit V1	Wi-Fi microcontroller
Hardware	Soil-moisture, rain, water-level, LDR	Analog field sensing
Hardware	DS18B20 / DHT11	Soil and air temperature, humidity
Hardware	16×2 I²C LCD	On-site display
Hardware	5V relay + submersible pump	Automated irrigation actuation
Firmware	Arduino IDE (C/C++)	ESP32 programming
Firmware	DHT, OneWire, DallasTemperature, LiquidCrystal_I2C	Sensor and LCD libraries

Backend	Python 3 / Flask	Irrigation logic and REST API
Database	Firebase Realtime Database	Live and historical data storage
Frontend	HTML5, CSS3, JavaScript (ES6 modules), Chart.js	Web dashboard, live sensor cards, and historical charts
Tooling	Git, VS Code	Version control and development

5.5 Module Implementation

5.5.1 Sensor-Node Firmware

The ESP32 firmware initializes the Wi-Fi connection, the DHT11, the DS18B20 (via the OneWire and DallasTemperature libraries), and the I²C LCD, then every 10 seconds reads all six sensors, refreshes the LCD, publishes the readings as a JSON object to Firebase, and applies any pending pump command to the relay. A simplified excerpt of the main loop is shown below:

```
void loop() {
  float soilMoisture = readPercent(SOIL_PIN);   // GPIO34
  float rainLevel    = readPercent(RAIN_PIN);   // GPIO35
  float waterLevel   = readPercent(WATER_PIN);  // GPIO33
  int   lightLevel   = analogRead(LDR_PIN);     // GPIO32
  sensors.requestTemperatures();
  float soilTemp     = sensors.getTempCByIndex(0);  // DS18B20 GPIO27
  float temperature  = dht.readTemperature();       // DHT11 GPIO4
  float humidity     = dht.readHumidity();

  showOnLCD(soilMoisture, temperature, humidity);
  publishReading(soilMoisture, rainLevel, waterLevel,
                 lightLevel, soilTemp, temperature, humidity);
  applyPumpCommand();   // relay on GPIO26
  delay(10000);
}
```

Each analog sensor's raw value (0–4095 on the ESP32's 12-bit ADC) is mapped to a 0–100% scale using calibration constants obtained by recording the sensor's output at its dry and wet extremes, as described in Appendix A.

5.5.2 Irrigation-Control Module

A Flask background job periodically reads the latest soil-moisture value and the configured threshold from Firebase, and writes a pump ON/OFF command back to the database when the reading crosses the threshold, with a small hysteresis band to avoid rapidly toggling the pump near the threshold value. If the manualOverride flag is set, the automatic logic is skipped and the pump follows the user's manual command instead.

```
def check_and_control_irrigation(reading, config):
    if config.get("manualOverride"):
        return  # user has taken manual control
    moisture   = reading["soilMoisture"]
    threshold  = config["moistureThreshold"]
    hysteresis = config.get("hysteresis", 5)
```

```
    if moisture < threshold - hysteresis:
        set_pump_state(True)
    elif moisture > threshold + hysteresis:
        set_pump_state(False)
```

5.5.3 LCD Display Module

The showOnLCD() routine formats the most important readings — soil moisture, air temperature, and humidity — onto the two 16-character rows of the I²C LCD, and cycles additional values (rain, water level, soil temperature, and pump state) on a short timer so that all parameters are visible on site without any network connection.

5.5.4 Dashboard Module

The React dashboard subscribes to Firebase in real time, rendering the latest readings as summary cards and the historical data as line charts using Chart.js. A settings panel lets the user update the irrigation threshold and safe alert ranges, and a toggle switch lets the user enable manual override and directly control the pump.

5.5.5 Alerting and Authentication

A simple rule-based checker compares each new reading against the safe minimum/maximum ranges stored in config and flags an alert banner on the dashboard when a reading falls outside range, along with a time-stamped entry written to the alerts node. Dashboard access is protected using Firebase Authentication with email/password sign-in, satisfying non-functional requirement NFR5 and functional requirement FR11.

5.6 Implementation Challenges

Three challenges stood out during development. First, because the ESP32 exposes several input-only ADC pins and shares ADC2 with Wi-Fi, the analog sensors had to be assigned carefully to ADC1 pins (GPIO32–35) so that readings remained stable while Wi-Fi was active. Second, intermittent Wi-Fi drops in the test environment occasionally caused missed uploads; this was mitigated by buffering the last few readings in memory and resending them once the connection was restored, satisfying NFR2, while the LCD continued to show live values throughout. Third, an early version of the irrigation logic toggled the pump rapidly when the soil-moisture value hovered close to the threshold; introducing the hysteresis band described in Section 5.5.2 resolved this.

5.7 Summary

With the ESP32 sensor node, on-site LCD, cloud database, backend irrigation logic, dashboard, and authentication all implemented and integrated, the system was ready for the testing and evaluation described in Chapter 6.

CHAPTER 6

Testing and Results

This chapter presents how the AgroSense system was tested and evaluates its performance against the requirements defined in Chapter 3.

6.1 Overview

Testing was carried out at three levels: unit testing of individual modules, integration testing of the sensor-to-cloud-to-dashboard pipeline, and field-style testing of the assembled prototype over a 7-day period under varying soil-moisture conditions.

6.2 Testing Strategy

Unit testing covered individual firmware functions (analog scaling, DHT11 and DS18B20 reads, LCD formatting) and backend functions (threshold comparison, hysteresis logic, pump-command logic) using manual test scripts and Python's unittest framework for the backend. Integration testing verified that data published by the ESP32 correctly appeared in Firebase, on the LCD, and on the dashboard within the required time window, and that configuration changes made on the dashboard correctly propagated back to the backend's irrigation logic. Field-style testing placed the prototype in a small potted-soil test bed and monitored it continuously over a 7-day period, including manually varying soil moisture to confirm that the automated irrigation logic responded correctly.

6.3 Test Cases

Table 6.1 summarizes representative test cases and their outcomes, covering all six sensors, the LCD, cloud communication, irrigation control, alerting, authentication, and dashboard responsiveness.

Table 6.1: Sample test cases and results.

ID	Test Case	Input	Expected Output	Result
T1	Soil-moisture accuracy (dry)	Sensor in dry soil	Reading below 30%	Pass
T2	Soil-moisture accuracy (wet)	Sensor in wet soil	Reading above 70%	Pass
T3	Rain-sensor detection	Water on sensor	Rain level rises above 60%	Pass
T4	Water-level sensor	Probe in filled tank	Level rises toward 100%	Pass
T5	LDR light reading	Cover / uncover sensor	Light value falls / rises	Pass

T6	DS18B20 soil temperature	Warm / cool probe	Temperature tracks change	Pass
T7	DHT11 temp / humidity	Breathe on sensor	Humidity rises	Pass
T8	LCD display	Node powered on	All readings shown and cycled	Pass
T9	Cloud-upload latency	New reading	Visible in Firebase within 5 s	Pass
T10	Automated irrigation trigger	Moisture below threshold	Pump turns ON	Pass
T11	Automated irrigation cutoff	Moisture above threshold	Pump turns OFF	Pass
T12	Manual override	User toggles pump	Pump state changes immediately	Pass
T13	Wi-Fi reconnection	Wi-Fi dropped then restored	Buffered readings resynced	Pass
T14	Out-of-range alert	Temperature above safe limit	Alert banner shown	Pass
T15	Hysteresis behaviour	Moisture near threshold	Pump does not toggle within band	Pass
T16	Unauthorized access	Request without login	Access denied, redirected to login	Pass
T17	Dashboard responsiveness	Opened on mobile screen	Layout adapts correctly	Pass

All 17 representative test cases passed, with no critical defects observed during the 7-day field-style testing period.

6.4 Results and Output

The final dashboard successfully displayed live sensor readings updating within the required 5-second window, along with 24-hour and 7-day historical charts, while the LCD showed the same values on site. Figure 6.1 shows a representative 24-hour soil-moisture trend recorded from the test bed, with the dashed line marking the configured irrigation threshold; the sharp rises correspond to automated irrigation events.

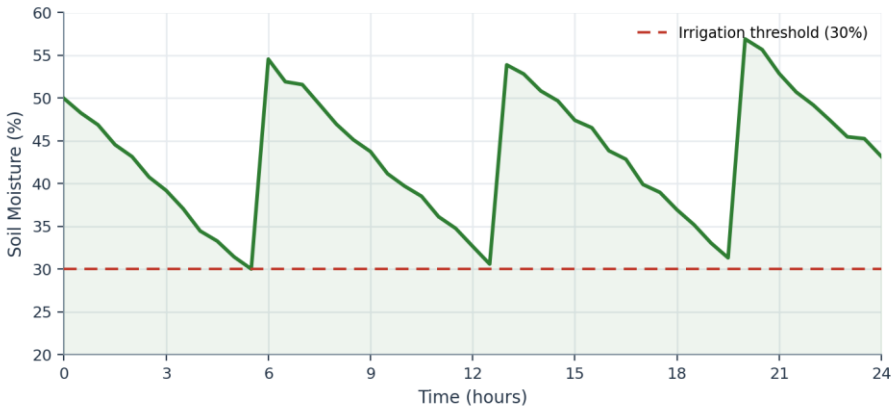


Figure 6.1: Soil-moisture trend over a 24-hour test period, showing automated irrigation events triggered at the 30% threshold.

Figure 6.2 shows the corresponding air temperature and humidity trend over the same 24-hour period, illustrating the typical diurnal pattern captured by the DHT11 sensor.

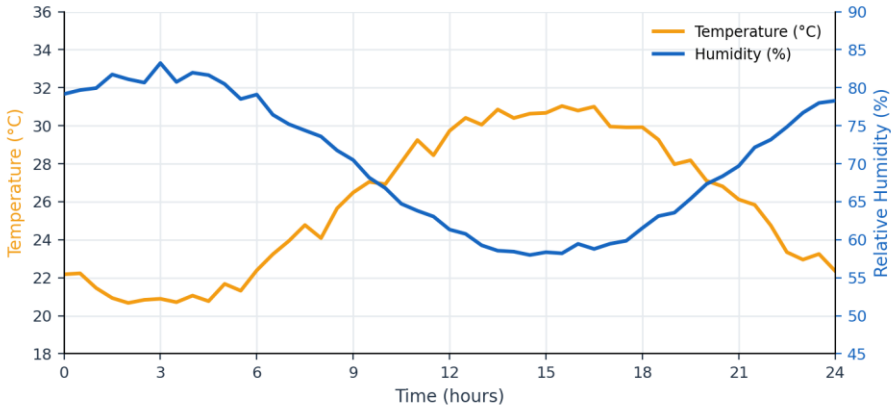


Figure 6.2: Air temperature and humidity trend recorded over the same 24-hour test period (DHT11).

During field-style testing, the automated irrigation module correctly activated the pump each time soil moisture dropped below the configured 30% threshold and deactivated it once moisture rose above the threshold with the hysteresis band applied, with no false triggers observed. The alerting module correctly flagged out-of-range temperature and humidity readings during simulated abnormal conditions, and the rain and water-level sensors behaved as expected when wetted.

6.5 Evaluation

Against the functional requirements defined in Chapter 3, all eleven requirements (FR1–FR11) were demonstrated as working in the test cases above. Figure 6.3 compares the number of watering events recorded over the same 7-day test period under a fixed-schedule

irrigation baseline versus the threshold-based automated irrigation implemented in this project.

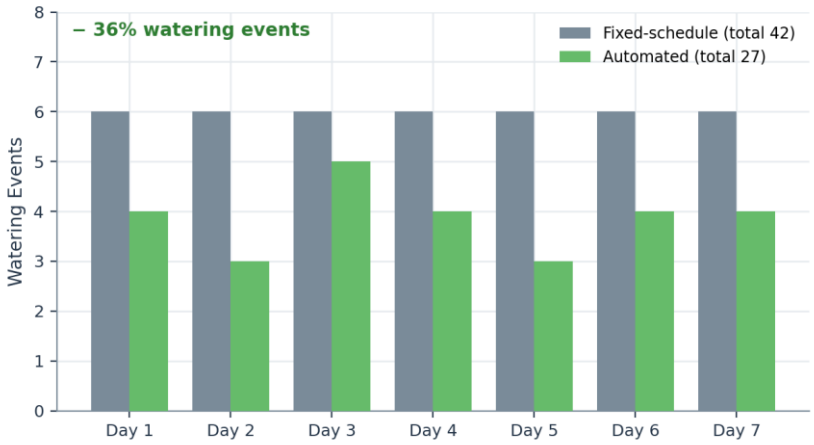


Figure 6.3: Number of watering events over a 7-day test period: fixed-schedule versus automated threshold-based irrigation.

As shown in Figure 6.3, automated irrigation reduced the number of watering events by roughly 36% (from 42 to 27 over the 7-day period) while keeping soil moisture within the target range more consistently, supporting the project's core motivation of reducing water waste. The main limitation observed was the sensitivity of the capacitive soil-moisture sensor to placement depth, discussed further in Chapter 7.

6.6 User Feedback Summary

In addition to automated testing, informal feedback was gathered from four test users who interacted with the dashboard and LCD prototype and answered the short questionnaire in Appendix C.2. Table 6.2 summarizes their average ratings on a 1–5 scale.

Table 6.2: Summary of informal user feedback (n = 4, scale 1–5).

Question	Avg. Rating
Ease of understanding field condition from the Home screen	4.5
Usefulness of the on-site LCD readout	4.6
Usefulness of historical trend charts	4.3
Clarity of threshold-configuration process	4.0
Helpfulness of alert banners	4.5

Overall, users found the on-site LCD, the colour-coded summary cards, and the alert banners the most immediately useful features, and requested SMS notifications for a future version — a suggestion incorporated into the future recommendations in Chapter 7.

6.7 Summary

Testing confirmed that the system meets its functional and non-functional requirements and that automated, sensor-driven irrigation performs measurably better than a fixed-schedule baseline, supporting the conclusions and future recommendations presented in the next chapter.

CHAPTER 7

Conclusion and Future Work

7.1 Conclusion

This project set out to design and build AgroSense, a low-cost IoT-based smart farming and crop monitoring system built around the ESP32 that gives farmers real-time visibility into field conditions and automates basic irrigation decisions. The completed prototype successfully collects soil moisture, rainfall, water level, light, soil temperature, and air temperature and humidity from an ESP32 sensor node, shows them on an on-site LCD, streams them to a cloud database, presents them through a live and historical web dashboard, and automatically controls irrigation based on a configurable soil-moisture threshold with manual override, while generating alerts for abnormal conditions. Testing confirmed that all eleven functional requirements were met, and that the automated irrigation approach reduced unnecessary watering by roughly 36% compared with a fixed-schedule baseline over a 7-day field-style test, within a hardware budget of under BDT 3,000.

Beyond the technical outcome, the project demonstrates that a genuinely usable smart-farming tool does not require expensive commercial hardware or subscriptions: a small, well-integrated stack of an ESP32, a handful of sensors, an LCD, a free-tier cloud database, and a simple web dashboard is sufficient to deliver meaningful value to a small or medium-scale farmer.

7.2 Limitations

The current prototype has several limitations. It supports only a single sensor node and does not yet scale to monitoring multiple zones within a larger field. The capacitive soil-moisture sensor's readings are sensitive to installation depth and soil type, which can affect calibration accuracy across different fields. The system depends on a stable Wi-Fi connection for cloud communication and does not yet support offline-first operation beyond the on-site LCD. It was evaluated on a small potted-soil test bed over a 7-day period rather than a full growing season on an actual working farm. Finally, the alerting system currently only supports in-dashboard banners; it does not yet push notifications to a farmer's phone when they are away from the dashboard.

These limitations are characteristic of an early single-node prototype rather than fundamental flaws in the approach. Each of them is addressed as a concrete, tractable item in the future-recommendations section that follows, and none requires a change to the core three-tier design established in Chapter 4.

7.3 Future Recommendations

Future work could extend the system in several directions:

- **Multi-zone monitoring.** Support multiple ESP32 nodes across different zones of a field, each reporting to the same dashboard, with per-zone thresholds and irrigation control.
- **Weather integration.** Integrate a weather-forecast API so the irrigation logic can factor in expected rainfall (complementing the on-board rain sensor) rather than relying solely on current soil moisture.
- **Long-range connectivity.** Add a low-power option such as LoRa to extend usability to fields without reliable Wi-Fi coverage.
- **Solar power.** Pair the ESP32 node with a small solar panel and battery to remove its dependence on a fixed power supply.
- **Push notifications.** Add SMS or push notifications for alerts so farmers are informed of urgent conditions even when away from the dashboard.
- **Pest and disease detection.** Incorporate image-based pest and disease detection using a camera module and a lightweight machine-learning model, and run a longer field trial across a full growing season.

7.4 Concluding Remarks

AgroSense demonstrates, end to end, that affordable and locally available components can be combined into a coherent system that measurably improves how a small farm is monitored and irrigated. The value of the project lies not in any single sophisticated component but in the careful integration of simple parts — an ESP32, six inexpensive sensors, an LCD, a free-tier cloud database, and a clear dashboard — into something a non-technical farmer could actually use.

REFERENCES

[1] S. Rahman and T. Islam, "IoT-Based Smart Irrigation Systems: A Review of Architectures and Applications," International Journal of Computer Applications, vol. 186, no. 4, pp. 12–19, 2024.

[2] M. Chowdhury and N. Akter, "Design of a Low-Cost Sensor Network for Precision Agriculture in Developing Regions," in Proceedings of the International Conference on Innovation in Engineering and Technology, pp. 1–6, 2023.

[3] R. Karim, "Cloud-Based Real-Time Data Logging for Embedded IoT Applications," Journal of Emerging Technologies in Computing, vol. 9, no. 2, pp. 45–53, 2022.

[4] Espressif Systems, ESP32 Technical Reference Manual, Version 5.1, Shanghai, China: Espressif Systems, 2023.

[5] Maxim Integrated, DS18B20 Programmable Resolution 1-Wire Digital Thermometer Datasheet, Rev. 3, Sunnyvale, CA: Maxim Integrated, 2019.

[6] Aosong Electronics, DHT11 Temperature and Humidity Sensor Datasheet, 2020.

[7] Google LLC, Firebase Realtime Database Documentation, 2024. [Online]. Available: https://firebase.google.com/docs/database.

[8] Facebook Inc., React — A JavaScript Library for Building User Interfaces, 2024. [Online]. Available: https://react.dev.

[9] Chart.js Contributors, Chart.js Documentation, v4, 2024. [Online]. Available: https://www.chartjs.org.

[10] P. Patel and D. Shah, "A Survey of Low-Power Wide-Area Networks for Agricultural IoT," Sensors and Actuators Reports, vol. 5, pp. 100–112, 2023.

[11] United Nations, Sustainable Development Goal 6: Clean Water and Sanitation, 2023. [Online]. Available: https://sdgs.un.org/goals/goal6.

[12] Food and Agriculture Organization, The State of Food and Agriculture: Water Challenges in Farming, Rome: FAO, 2022.

APPENDIX A

A.1 Hardware Setup Guide

Follow these steps to assemble and wire the AgroSense ESP32 sensor node. Refer to Figure 5.1 for the complete wiring diagram.

1. Connect the capacitive soil-moisture sensor's AO pin to GPIO34 of the ESP32. Power the sensor from the 3.3 V rail.
2. Connect the rain sensor's AO pin to GPIO35. Power from 3.3 V.
3. Connect the water-level sensor's AO pin to GPIO33. Power from 3.3 V.
4. Connect the LDR module's AO pin to GPIO32. Power from 3.3 V.
5. Connect the DS18B20 DQ pin to GPIO27. Place a 4.7 kΩ resistor between DQ and 3.3 V (pull-up). Power the sensor from 3.3 V.
6. Connect the DHT11 DATA pin to GPIO4. Place a 10 kΩ resistor between DATA and 3.3 V (pull-up). Power the sensor from 3.3 V.
7. Connect the I²C LCD SDA to GPIO21 and SCL to GPIO22. Power the LCD from the 5 V rail (VIN on the ESP32 board).
8. Connect the relay module IN to GPIO26. Power the relay from 5 V. Connect the pump through the relay's Normally-Open (NO) and Common (COM) contacts to a separate 5 V or 12 V external supply.
9. Join all module GND pins, the ESP32 GND, and the external supply GND to a common ground. Never connect 5 V to any ESP32 GPIO.
10. Power the ESP32 from a 5 V 2 A USB adapter. Confirm the Wi-Fi LED blinks before proceeding.

A.2 Sensor Calibration Procedure

The analog sensors (soil moisture, rain, and water level) output a raw 12-bit ADC value (0–4095) on the ESP32. Map this to a 0–100% scale using the following calibration constants:

- Soil moisture: record the raw value in fully dry air (DRY_VAL ≈ 3200) and in fully saturated soil (WET_VAL ≈ 1200). Percentage = ((DRY_VAL − raw) / (DRY_VAL − WET_VAL)) × 100, clamped to [0, 100].

- Rain sensor: record the raw value in dry conditions (DRY ≈ 4000) and with water on the sensor (WET ≈ 500). Apply the same mapping formula.
- Water level: calibrate against known depth markings on the sensor, mapping the raw ADC range to 0–100%.
- DS18B20 and DHT11 return calibrated values in °C and % RH directly from their libraries; no further calibration is required.

A.3 Firebase Configuration

Create a Firebase project, enable the Realtime Database in test mode for development, and copy the database URL and web API key into the firmware configuration header (config.h) and the Flask backend environment variables. Enable Firebase Authentication and add one email/password account for dashboard access.

APPENDIX B

This appendix presents the complete source-code listings for the ESP32 sensor-node firmware and the Flask backend irrigation service.

B.1 Complete ESP32 Firmware

```
#include <WiFi.h>
#include <time.h>
#include <FirebaseESP32.h>
#include <DHT.h>
#include <OneWire.h>
#include <DallasTemperature.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <addons/TokenHelper.h>
#include <addons/RTDBHelper.h>

// ── WiFi / Firebase CREDENTIALS ──────────────────────────
#define WIFI_SSID      "masum"
#define WIFI_PASSWORD  "123456789"
#define API_KEY        "AIzaSyAgMzY-3nDHQ2BJkPDkY7dUiGSDV6wefag"
#define DATABASE_URL   "https://smart-farming-3d1f2-default-rtdb.asia-
southeast1.firebasedatabase.app"
#define USER_EMAIL     "agrosense@gmail.com"
#define USER_PASSWORD  "agrosense123"

// NTP settings (for correct epoch timestamp)
#define NTP_SERVER     "pool.ntp.org"
#define GMT_OFFSET_SEC 0   // MUST be 0 (UTC) to match web frontend
Date.now()
#define DST_OFFSET_SEC 0

// ── GPIO PINS ────────────────────────────────────────────────────
#define PIN_SOIL_MOISTURE  34
#define PIN_RAIN           35
#define PIN_WATER_LEVEL    33
#define PIN_LDR            32
#define PIN_DS18B20        27
#define PIN_DHT11          4
#define PIN_RELAY_PUMP     26
#define OLED_SDA           21
#define OLED_SCL           22
```

```
// ── OLED CONFIG ──────────────────────────────────────────────────────
#define OLED_WIDTH   128
#define OLED_HEIGHT   64
#define OLED_ADDR   0x3C    // Try 0x3D if display blank

// ── ADC CALIBRATION ──────────────────────────────────────────────────
// All resistive sensors: HIGH ADC = dry/no-contact, LOW ADC = wet/submerged
// ► To calibrate: read Serial Monitor "raw:" values in dry and wet states
//
// Soil Moisture  (GPIO34): dry air ≈ 3800, wet soil ≈ 1200
#define SOIL_DRY     3800
#define SOIL_WET     1200

// Rain Sensor    (GPIO35): dry PCB ≈ 4000, submerged ≈ 800
#define RAIN_DRY     4000
#define RAIN_WET      800

// Water Level    (GPIO33): no water/floating ≈ 3900, full tank ≈ 600
// INVERTED: high ADC = empty (0%), low ADC = full (100%)
#define WATER_EMPTY  3900
#define WATER_FULL    600

// LDR Light      (GPIO32): dark ≈ 4095, bright light ≈ 0
#define LDR_DARK     4095
#define LDR_BRIGHT      0
#define LUX_MAX      1200

// ── TIMING ───────────────────────────────────────────────────────────
#define SENSOR_INTERVAL   2000    // ms — read & push sensor data
#define DEVICE_INTERVAL  10000    // ms — push device telemetry
#define OLED_INTERVAL     3000    // ms — cycle OLED display page
#define WIFI_CHECK_MS     8000    // ms — WiFi watchdog interval (8s for fast recovery)

// ── SENSOR OBJECTS ───────────────────────────────────────────────────
DHT               dht(PIN_DHT11, DHT11);
OneWire           oneWire(PIN_DS18B20);
DallasTemperature ds18b20(&oneWire);
Adafruit_SSD1306  oled(OLED_WIDTH, OLED_HEIGHT, &Wire, -1);

// ── FIREBASE OBJECTS ─────────────────────────────────────────────────
```

```
FirebaseData   fbdo;
FirebaseAuth   fbauth;
FirebaseConfig fbconfig;

// ── STATE ──────────────────────────────────────────────────────────────
unsigned long lastSensorMs  = 0;
unsigned long lastDeviceMs  = 0;
unsigned long lastOledMs    = 0;
unsigned long lastWifiCheck = 0;
unsigned long startMs       = 0;
unsigned long uptimeSeconds = 0;
bool          wifiJustReconnected = false;  // flag: push immediately after
reconnect
int           oledPage      = 0;
bool          pumpStatus    = false;
String        pumpMode      = "manual";
bool          firebaseInited = false;

// ── SENSOR READINGS ────────────────────────────────────────────────────
float tempC         = 0;
float humidPct      = 0;
float soilMoistPct  = 0;
float soilTempC     = 0;
float rainLevelPct  = 0;
float lightLux      = 0;
float waterLevelPct = 0;

//
──────────────────────────────────────────────────────────────────────────
──
// HELPER: mapFloat — maps float value from one range to another
//
──────────────────────────────────────────────────────────────────────────
──
float mapFloat(float val, float inMin, float inMax, float outMin, float
outMax) {
  float result = (val - inMin) * (outMax - outMin) / (inMax - inMin) +
outMin;
  return constrain(result, min(outMin, outMax), max(outMin, outMax));
}

//
──────────────────────────────────────────────────────────────────────────
──
// HELPER: epochSec — returns current epoch time in seconds (from NTP)
// Returns millis() / 1000 as fallback if NTP not synced
```

```
//
──────────────────────────────────────────────────────────────────────────
──
long epochSec() {
  struct timeval tv;
  gettimeofday(&tv, nullptr);
  if (tv.tv_sec < 1000000000LL) {
    // NTP not synced — use millis as approximate seconds
    return millis() / 1000;
  }
  return tv.tv_sec;
}

//
──────────────────────────────────────────────────────────────────────────
──
// OLED HELPER: show full-screen status message
//
──────────────────────────────────────────────────────────────────────────
──
void oledStatus(const String& line1, const String& line2 = "", const
String& line3 = "") {
  oled.clearDisplay();
  oled.setTextColor(SSD1306_WHITE);
  oled.setTextSize(1);
  oled.setCursor(0, 0);  oled.println(line1);
  oled.setCursor(0, 20); oled.println(line2);
  oled.setCursor(0, 40); oled.println(line3);
  oled.display();
}

//
──────────────────────────────────────────────────────────────────────────
──
// WiFi CONNECT — up to 3 attempts × 10 s each
// Returns true if connected
//
──────────────────────────────────────────────────────────────────────────
──
bool connectWiFi() {
  if (WiFi.status() == WL_CONNECTED) return true;

  Serial.println("\n[WiFi] Connecting to: " + String(WIFI_SSID));
  WiFi.mode(WIFI_STA);
  WiFi.disconnect(false);
  delay(200);

  for (int attempt = 1; attempt <= 3; attempt++) {
```

```
    Serial.printf("[WiFi] Attempt %d/3...\n", attempt);
    oledStatus("WiFi Connecting...",
               "SSID: " + String(WIFI_SSID),
               "Attempt " + String(attempt) + "/3");

    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

    unsigned long t = millis();
    while (WiFi.status() != WL_CONNECTED && millis() - t < 10000) {  // 10s
per attempt
      delay(300);
      Serial.print(".");
    }
    Serial.println();

    if (WiFi.status() == WL_CONNECTED) {
      String ip = WiFi.localIP().toString();
      int rssi   = WiFi.RSSI();
      Serial.printf("[WiFi] Connected! IP: %s  RSSI: %d dBm\n", ip.c_str(),
rssi);

      // Sync NTP time (Crucial for Firebase SSL/Token)
      configTime(GMT_OFFSET_SEC, DST_OFFSET_SEC, NTP_SERVER);
      Serial.println("[NTP] Time sync requested...");

      // Wait until time is synced (epoch > 100000)
      time_t now = time(nullptr);
      int ntpRetry = 0;
      while (now < 100000 && ntpRetry < 50) {
        delay(100);
        now = time(nullptr);
        ntpRetry++;
      }
      if (now > 100000) Serial.println("[NTP] Time synced successfully!");
      else Serial.println("[NTP] Time sync timeout, proceeding anyway.");

      oled.clearDisplay();
      oled.setTextSize(1); oled.setTextColor(SSD1306_WHITE);
      oled.setCursor(0,  0); oled.println("  WiFi Connected!");
      oled.setCursor(0, 16); oled.println("IP: " + ip);
      oled.setCursor(0, 32); oled.println("RSSI: " + String(rssi) + "
dBm");
      oled.setCursor(0, 48); oled.println("AgroSense Online");
      oled.display();
      delay(1000);  // short delay — don't block too long
      wifiJustReconnected = true;  // trigger immediate push
      return true;
    }
```

```
    Serial.printf("[WiFi] Attempt %d failed. Waiting 2s...\n", attempt);
    WiFi.disconnect(true);
    delay(2000);
  }

  Serial.println("[WiFi] All attempts failed. Running OFFLINE.");
  oledStatus("WiFi FAILED!", "Running Offline", "Sensors Active");
  delay(2000);
  return false;
}

//
──────────────────────────────────────────────────────────────────────────────
──
// FIREBASE INIT
//
──────────────────────────────────────────────────────────────────────────────
──
void initFirebase() {
  if (WiFi.status() != WL_CONNECTED) return;
  fbconfig.api_key               = API_KEY;
  fbconfig.database_url          = DATABASE_URL;
  fbauth.user.email              = USER_EMAIL;
  fbauth.user.password           = USER_PASSWORD;
  fbconfig.token_status_callback = tokenStatusCallback;
  Firebase.begin(&fbconfig, &fbauth);
  Firebase.reconnectWiFi(true);
  firebaseInited = true;
  Serial.println("[Firebase] Initialized.");
}

//
──────────────────────────────────────────────────────────────────────────────
──
// SETUP
//
──────────────────────────────────────────────────────────────────────────────
──
void setup() {
  Serial.begin(115200);
  delay(500);
  Serial.println("\n=========================================");
  Serial.println("     AgroSense Smart Farming  v2.0");
  Serial.println("=========================================");

  // Pump relay — ensure OFF at boot
  pinMode(PIN_RELAY_PUMP, OUTPUT);
```

```
  digitalWrite(PIN_RELAY_PUMP, LOW);

  // Sensors
  dht.begin();
  ds18b20.begin();

  // OLED init
  Wire.begin(OLED_SDA, OLED_SCL);
  if (!oled.begin(SSD1306_SWITCHCAPVCC, OLED_ADDR)) {
    Serial.println("[OLED] Not found at 0x3C! Check wiring or try 0x3D.");
    // Continue without OLED — non-fatal
  } else {
    oled.clearDisplay();
    oled.setTextSize(2); oled.setTextColor(SSD1306_WHITE);
    oled.setCursor(10, 10); oled.println("AgroSense");
    oled.setTextSize(1);
    oled.setCursor(20, 40); oled.println("Smart Farming v2");
    oled.display();
    delay(2000);
  }

  // WiFi — connect with auto-retry
  bool wifiOK = connectWiFi();

  // Firebase — only if WiFi is up
  if (wifiOK) {
    initFirebase();
    delay(2000);  // let auth token settle
    if (Firebase.ready()) {
      Firebase.setBool(fbdo,   "SmartFarming/Device/online",    true);
      Firebase.setString(fbdo, "SmartFarming/Device/ipAddress",
WiFi.localIP().toString());
      Firebase.setInt(fbdo,    "SmartFarming/Device/uptime",    0);
      Firebase.setInt(fbdo,    "SmartFarming/Device/wifiRSSI",  WiFi.RSSI()
);
      Serial.println("[Firebase] Initial device status OK.");
    }
  } else {
    Serial.println("[Firebase] Skipped — no WiFi.");
  }

  startMs = millis();
  Serial.println("[SmartFarm] Loop started. Sensor interval: " +
String(SENSOR_INTERVAL) + "ms");
}
```

```
//
──────────────────────────────────────────────────────────────────────────────
──
// LOOP
//
──────────────────────────────────────────────────────────────────────────────
──
void loop() {
  unsigned long now = millis();
  uptimeSeconds = (now - startMs) / 1000;

  // ── WiFi watchdog: check & reconnect every 8 s
──────────────────────────
  if (now - lastWifiCheck >= WIFI_CHECK_MS) {
    lastWifiCheck = now;
    if (WiFi.status() != WL_CONNECTED) {
      Serial.println("[WiFi] Lost connection! Reconnecting...");
      bool ok = connectWiFi();
      if (ok && !firebaseInited) initFirebase();
    }
  }

  // ── Immediate push after reconnect
──────────────────────────────────────
  if (wifiJustReconnected && WiFi.status() == WL_CONNECTED) {
    // Let Firebase-ESP-Client handle the reconnection internally.
    // Wait up to 10 seconds for auth token to become ready
    unsigned long fbWait = millis();
    while (!Firebase.ready() && millis() - fbWait < 10000) {
      delay(300);
    }

    if (Firebase.ready()) {
      wifiJustReconnected = false;
      Serial.println("[WiFi] Reconnected + Firebase ready — pushing
immediately.");
      readAllSensors();
      pushSensors();
      pushDevice();
      lastSensorMs = now;
      lastDeviceMs = now;
    } else {
      // Still not ready — try next cycle
      Serial.println("[Firebase] Token still not ready, will retry normally
in loop.");
      wifiJustReconnected = false; // Don't block the loop forever
    }
  }
```

```
  // ── Read + push sensors every SENSOR_INTERVAL ──────────────────────
  if (now - lastSensorMs >= SENSOR_INTERVAL) {
    lastSensorMs = now;
    readAllSensors();
    if (WiFi.status() == WL_CONNECTED && Firebase.ready()) pushSensors();
  }

  // ── Push device telemetry every DEVICE_INTERVAL ────────────────────
  if (now - lastDeviceMs >= DEVICE_INTERVAL) {
    lastDeviceMs = now;
    if (WiFi.status() == WL_CONNECTED && Firebase.ready()) {
      pushDevice();
      checkPumpCommand();
    }
  }

  // ── Cycle OLED display every OLED_INTERVAL ─────────────────────────
  if (now - lastOledMs >= OLED_INTERVAL) {
    lastOledMs = now;
    updateOLED();
  }
}

//
──────────────────────────────────────────────────────────────────────────
──
// READ ALL SENSORS
//
──────────────────────────────────────────────────────────────────────────
──
void readAllSensors() {

  // ── DHT11: Temperature & Humidity (retry up to 3× on NaN) ──────────
  float t = NAN, h = NAN;
  for (int retry = 0; retry < 3 && (isnan(t) || isnan(h)); retry++) {
    if (retry > 0) delay(300);
    t = dht.readTemperature();
    h = dht.readHumidity();
  }
  if (!isnan(t) && t >= -10.0 && t <= 80.0) tempC    = t;
  if (!isnan(h) && h >=   0.0 && h <= 100.0) humidPct = h;
```

```
  // ── DS18B20: Soil Temperature ──────────────────────────────
  ds18b20.requestTemperatures();
  float st = ds18b20.getTempCByIndex(0);
  if (st != DEVICE_DISCONNECTED_C && st >= -55.0 && st <= 125.0) soilTempC
= st;

  // ── Analog sensors: average 5 samples for noise reduction ──
  long rawSoil = 0, rawRain = 0, rawWater = 0, rawLight = 0;
  for (int i = 0; i < 5; i++) {
    rawSoil  += analogRead(PIN_SOIL_MOISTURE);
    rawRain  += analogRead(PIN_RAIN);
    rawWater += analogRead(PIN_WATER_LEVEL);
    rawLight += analogRead(PIN_LDR);
    delay(10);
  }
  rawSoil /= 5; rawRain /= 5; rawWater /= 5; rawLight /= 5;

  // ── Map ADC → meaningful units ─────────────────────────────
  soilMoistPct  = mapFloat(rawSoil,  SOIL_DRY,    SOIL_WET,    0, 100);
  rainLevelPct  = mapFloat(rawRain,  RAIN_DRY,    RAIN_WET,    0, 100);
  waterLevelPct = mapFloat(rawWater, WATER_EMPTY, WATER_FULL,  0, 100);  //
inverted
  lightLux      = mapFloat(rawLight, LDR_DARK,    LDR_BRIGHT,  0, LUX_MAX);

  // ── Debug output (raw values help you calibrate) ───────────
  Serial.printf(
    "[Sensors] T:%.1f°C H:%.0f%% Soil:%.0f%%(r:%ld) SoilT:%.1f°C "
    "Rain:%.0f%%(r:%ld) Light:%.0flux(r:%ld) Water:%.0f%%(r:%ld)\n",
    tempC, humidPct,
    soilMoistPct, rawSoil, soilTempC,
    rainLevelPct, rawRain,
    lightLux,     rawLight,
    waterLevelPct,rawWater
  );
}

//
──────────────────────────────────────────────────────────────────────
──
// PUSH SENSORS → FIREBASE
// Key names MUST match web dashboard SENSOR_CONFIG IDs exactly
//
──────────────────────────────────────────────────────────────────────
──
```

```
void pushSensors() {
  FirebaseJson sensors;
  sensors.set("temperature",  tempC);
  sensors.set("humidity",     humidPct);
  sensors.set("soilMoisture", soilMoistPct);
  sensors.set("soilTemp",     soilTempC);     // web expects "soilTemp"
  sensors.set("rainLevel",    rainLevelPct);  // web expects "rainLevel"
  sensors.set("lightLevel",   lightLux);      // web expects "lightLevel"
  sensors.set("waterLevel",   waterLevelPct);

  if (!Firebase.updateNode(fbdo, "SmartFarming/Sensors", sensors)) {
    Serial.println("[Firebase] Sensor push error: " + fbdo.errorReason());
    return;
  }

  // Epoch timestamp in seconds (from NTP if synced, else millis/1000)
  Firebase.setInt(fbdo, "SmartFarming/Device/lastUpdate", (int)epochSec());
}

//
──────────────────────────────────────────────────────────────────────
──
// PUSH DEVICE TELEMETRY → FIREBASE
//
──────────────────────────────────────────────────────────────────────
──
void pushDevice() {
  FirebaseJson device;
  device.set("online",     true);
  device.set("uptime",     (int)uptimeSeconds);
  device.set("wifiRSSI",   (int)WiFi.RSSI());
  device.set("ipAddress",  WiFi.localIP().toString());
  device.set("lastUpdate", (int)epochSec());

  if (!Firebase.updateNode(fbdo, "SmartFarming/Device", device)) {
    Serial.println("[Firebase] Device push error: " + fbdo.errorReason());
  }
}

//
──────────────────────────────────────────────────────────────────────
──
// CHECK PUMP COMMAND FROM FIREBASE
//
──────────────────────────────────────────────────────────────────────
──
void checkPumpCommand() {
  // Read mode first
```

```
  if (Firebase.getString(fbdo, "SmartFarming/Pump/mode")) {
    pumpMode = fbdo.stringData();
  }

  if (pumpMode == "auto") {
    // Auto: pump ON when soil moisture < 30%
    bool shouldRun = (soilMoistPct < 30.0f);
    if (shouldRun != pumpStatus) {
      pumpStatus = shouldRun;
      digitalWrite(PIN_RELAY_PUMP, pumpStatus ? HIGH : LOW);
      Firebase.setBool(fbdo, "SmartFarming/Pump/status", pumpStatus);
      Serial.printf("[AutoPump] Soil=%.0f%% → Pump %s\n",
                    soilMoistPct, pumpStatus ? "ON" : "OFF");
    }
  } else {
    // Manual: obey Firebase command
    if (Firebase.getBool(fbdo, "SmartFarming/Pump/status")) {
      bool cmd = fbdo.boolData();
      if (cmd != pumpStatus) {
        pumpStatus = cmd;
        digitalWrite(PIN_RELAY_PUMP, pumpStatus ? HIGH : LOW);
        Serial.printf("[Pump] Manual → %s\n", pumpStatus ? "ON" : "OFF");
      }
    }
  }

  // Runtime counter
  static unsigned long pumpStartMs = 0;
  if (pumpStatus && pumpStartMs == 0) pumpStartMs = millis();
  if (!pumpStatus) pumpStartMs = 0;
  if (pumpStatus && pumpStartMs > 0) {
    int runtime = (int)((millis() - pumpStartMs) / 1000);
    Firebase.setInt(fbdo, "SmartFarming/Pump/runtime", runtime);
  }
}

//
──────────────────────────────────────────────────────────────────────
──
// OLED DISPLAY — cycles through 5 pages
//
──────────────────────────────────────────────────────────────────────
──
void drawLabel(int x, int y, const String& label, const String& val, const
String& unit) {
  oled.setCursor(x, y);
  oled.setTextSize(1); oled.setTextColor(SSD1306_WHITE);
  oled.print(label + ": ");
```

```
  oled.setTextSize(1);
  oled.print(val);
  oled.print(unit);
}

void updateOLED() {
  oled.clearDisplay();

  // Header bar
  oled.fillRect(0, 0, OLED_WIDTH, 10, SSD1306_WHITE);
  oled.setTextColor(SSD1306_BLACK);
  oled.setTextSize(1);

  switch (oledPage) {
    // ── Page 0: Temperature & Humidity
──────────────────────────────────
    case 0:
      oled.setCursor(28, 1); oled.print("TEMP & HUMIDITY");
      oled.setTextColor(SSD1306_WHITE);
      oled.setTextSize(2);
      oled.setCursor(10, 14); oled.printf("%.1f", tempC);
      oled.setTextSize(1); oled.print(" C");
      oled.setTextSize(2);
      oled.setCursor(10, 38); oled.printf("%.0f", humidPct);
      oled.setTextSize(1); oled.print(" %RH");
      break;

    // ── Page 1: Soil Moisture & Soil Temperature
──────────────────────────
    case 1:
      oled.setCursor(22, 1); oled.print("SOIL CONDITIONS");
      oled.setTextColor(SSD1306_WHITE);
      oled.setTextSize(1);
      oled.setCursor(0, 14); oled.printf("Moisture : %.0f %%",
soilMoistPct);
      oled.setCursor(0, 26); oled.printf("Soil Temp: %.1f C",  soilTempC);
      // Mini progress bar for soil moisture
      oled.drawRect(0, 42, 128, 10, SSD1306_WHITE);
      oled.fillRect(2, 44, (int)(soilMoistPct * 1.24f), 6, SSD1306_WHITE);
      oled.setCursor(0, 55); oled.printf("Dry 0%%        100%% Wet");
      break;

    // ── Page 2: Rain & Light
──────────────────────────────────────────
    case 2:
      oled.setCursor(30, 1); oled.print("RAIN & LIGHT");
      oled.setTextColor(SSD1306_WHITE);
      oled.setTextSize(1);
```

```
      oled.setCursor(0, 14); oled.printf("Rain  : %.0f
%%",     rainLevelPct);
      oled.setCursor(0, 26); oled.printf("Light : %.0f lux",    lightLux);
      // Mini progress bar for rain
      oled.drawRect(0, 42, 128, 10, SSD1306_WHITE);
      oled.fillRect(2, 44, (int)(rainLevelPct * 1.24f), 6, SSD1306_WHITE);
      oled.setCursor(0, 55); oled.print("No Rain     Heavy Rain");
      break;

    // ── Page 3: Water Level & Pump ──────────────────────
    case 3:
      oled.setCursor(24, 1); oled.print("WATER & PUMP");
      oled.setTextColor(SSD1306_WHITE);
      oled.setTextSize(1);
      oled.setCursor(0, 14); oled.printf("Water : %.0f
%%",   waterLevelPct);
      oled.setCursor(0, 26); oled.printf("Pump  : %s",        pumpStatus ?
"RUNNING" : "OFF");
      oled.setCursor(0, 38); oled.printf("Mode  :
%s",        pumpMode.c_str());
      // Water level bar
      oled.drawRect(0, 50, 128, 10, SSD1306_WHITE);
      oled.fillRect(2, 52, (int)(waterLevelPct * 1.24f), 6, SSD1306_WHITE);
      break;

    // ── Page 4: Device Status ───────────────────────────
    case 4: {
      oled.setCursor(28, 1); oled.print("DEVICE STATUS");
      oled.setTextColor(SSD1306_WHITE);
      oled.setTextSize(1);
      bool wifiUp = (WiFi.status() == WL_CONNECTED);
      oled.setCursor(0, 14); oled.printf("WiFi  : %s", wifiUp ?
WiFi.localIP().toString().c_str() : "Offline");
      oled.setCursor(0, 26); oled.printf("RSSI  : %d dBm",  wifiUp ?
WiFi.RSSI() : 0);
      oled.setCursor(0, 38); oled.printf("Uptime: %lu s",   uptimeSeconds);
      oled.setCursor(0, 50); oled.printf("FB    : %s", Firebase.ready() ?
"Connected" : "Offline");
      break;
    }
  }

  oled.display();
  oledPage = (oledPage + 1) % 5;
}
```

B.2 Flask Backend Irrigation Service

```python
import time, firebase_admin
from firebase_admin import db

def get_reading(): return db.reference("/latestReading").get()
def get_config():  return db.reference("/config").get()
def set_pump(state): db.reference("/config/pumpCommand").set(state)

def raise_alert(reading, reason):
    db.reference("/alerts").push({
        "timestamp": {".sv": "timestamp"},
        "reason": reason, "reading": reading })

def control(reading, config):
    if config.get("manualOverride"): return
    sm  = reading["soilMoisture"]
    thr = config["moistureThreshold"]
    hys = config.get("hysteresis", 5)
    if sm < thr - hys: set_pump(True)
    elif sm > thr + hys: set_pump(False)

def check_alerts(r, c):
    if not (c["tempSafeMin"] <= r["temperature"] <= c["tempSafeMax"]):
        raise_alert(r, "Temperature out of safe range")
    if not (c["humiditySafeMin"] <= r["humidity"] <= c["humiditySafeMax"]):
        raise_alert(r, "Humidity out of safe range")

def run():
    while True:
        r, c = get_reading(), get_config()
        if r and c: control(r, c); check_alerts(r, c)
        time.sleep(5)

if __name__ == "__main__": run()
```

APPENDIX C

This appendix provides a short user manual for operating the AgroSense dashboard and LCD, and the sample evaluation questionnaire used to gather informal feedback.

C.1 User Manual

11. Power on the sensor node. Connect the ESP32 to a 5 V 2 A USB adapter. The on-site LCD will display live readings once the node has connected to Wi-Fi.
12. Log in to the dashboard. Open the dashboard URL in a web browser or mobile browser and sign in with your registered email and password.
13. View live readings. The Home screen displays the latest values as colour-coded cards (green = normal, amber = warning, red = alert). The LCD shows the same values on site.
14. View trends. Open the Trends screen and select a 24-hour or 7-day range to view historical charts.
15. Configure irrigation threshold. Open the Settings screen, enter the desired soil-moisture threshold (the percentage below which irrigation starts), and tap Save.
16. Manually control the pump. On the Home screen, tap the pump toggle to enable manual override, then use the ON/OFF switch. Turn manual override off to return to automatic control.
17. Respond to alerts. If an alert banner appears, check the corresponding sensor reading and inspect the field if needed; the alert clears once the reading returns to the safe range.

C.2 Sample Evaluation Questionnaire

The following questionnaire was used to gather informal feedback from test users:

1. How easy was it to understand the current condition of the field from the Home screen? (1 = very difficult, 5 = very easy)
2. How useful did you find the on-site LCD display?
3. How useful did you find the historical trend charts?
4. Was the process of setting the irrigation threshold clear?
5. Did the alert banners help you notice abnormal conditions quickly?

6. Would you trust this system to control irrigation automatically on your own field? Why or why not?
7. What additional feature would make this system more useful to you?

APPENDIX D

This appendix provides the detailed bill of materials and the complete ESP32 pin-mapping table for the AgroSense prototype, supplementing the hardware setup guide in Appendix A and the wiring diagram in Figure 5.1.

D.1 Detailed Bill of Materials

Table D.1: Detailed bill of materials for the AgroSense prototype.

Item	Specification	Qty	Unit (BDT)	Total (BDT)
ESP32 DevKit V1	Dual-core 240 MHz, Wi-Fi + BT, 3.3 V	1	650	650
Capacitive soil-moisture sensor	Corrosion-resistant, analog out	1	180	180
Rain sensor module	Analog + digital out, 5 V coil	1	120	120
Water-level sensor	Analog out, 3.3 V	1	100	100
LDR module	Analog + digital out	1	60	60
DS18B20 waterproof probe	1-Wire, −55 to +125 °C, IP68	1	250	250
DHT11 sensor	0–50 °C, 20–90% RH	1	150	150
16×2 I²C LCD	PCF8574 backpack, addr 0x27	1	350	350
5 V relay module	Opto-isolated, 10 A rated	1	120	120
Submersible mini pump	5 V DC, ~120 L/h	1	350	350
5 V 2 A USB adapter	Power supply for ESP32	1	250	250
External pump PSU	5 V / 12 V depending on pump	1	200	200
10 kΩ resistor (×1)	DHT11 pull-up	1	5	5
4.7 kΩ resistor (×1)	DS18B20 pull-up	1	5	5
Jumper wires (×40)	Male-to-male and male-to-female	2 sets	50	100
Breadboard (830-tie)	For prototyping	1	90	90
Enclosure box	Weather-resistant ABS	1	150	150
Total				≈ 2,930

D.2 Complete ESP32 Pin-Mapping Reference

Table D.2: Complete pin-mapping for the AgroSense ESP32 sensor node.

ESP32 Pin	Connected To	Signal Type	Pull-up	Supply
GPIO34 (ADC1)	Soil-moisture AO	Analog in	None	3.3 V
GPIO35 (ADC1)	Rain sensor AO	Analog in	None	3.3 V
GPIO33 (ADC1)	Water-level AO	Analog in	None	3.3 V

GPIO32 (ADC1)	LDR module AO	Analog in	None	3.3 V
GPIO27	DS18B20 DQ	1-Wire digital	4.7 kΩ to 3.3 V	3.3 V
GPIO4	DHT11 DATA	Single-wire	10 kΩ to 3.3 V	3.3 V
GPIO21 (SDA)	LCD SDA	I²C data	Internal	5 V
GPIO22 (SCL)	LCD SCL	I²C clock	Internal	5 V
GPIO26	Relay IN	Digital out	None	5 V coil
3V3	Sensor VCC (×4)	Power	—	3.3 V
VIN / 5V	Relay VCC, LCD VCC	Power	—	5 V
GND	Common ground	Ground	—	—

Important: ADC2 pins (GPIO0, 2, 4, 12–15, 25–27 in some configurations) conflict with Wi-Fi and must not be used for analog sensor reads while Wi-Fi is active. All four analog sensors are assigned to ADC1 pins (GPIO32–35) to avoid this limitation.

BIOGRAPHY

of

Md Emam Masum



Md Emam Masum is a student of the Department of Computer Science and Engineering (CSE) at the Bangladesh Army University of Science and Technology (BAUST), Saidpur. He is particularly interested in embedded systems, cloud platforms, and data-driven applications. In this project, he played a key role in developing the Flask-based backend, designing the hardware architecture, and integrating sensors to ensure seamless data acquisition and communication. His contributions were instrumental in building a reliable and efficient system, reflecting his technical expertise and commitment to practical engineering solutions.

Project:

- **Title:** AgroSense: An IoT-Based Smart Farming and Crop Monitoring System.

Research:

- ***AI-Based Plagiarism Checker:*** *Worked on the development of an AI-based plagiarism detection system focused on identifying textual similarities and supporting automated content analysis.*
- ***IoT-Based Smart Farming:*** *Conducted research and development on an IoT-based agricultural monitoring system integrating ESP32, soil and environmental sensors, cloud connectivity, real-time monitoring, and automated irrigation.*

Md Emam Masum
+880 1774-109683
220201056@baust.edu.bd
Saidpur, Nilphamari, Bangladesh

BIOGRAPHY

of

Zihan Hossain



Zihan Hossain is a student of the Department of Computer Science and Engineering (CSE) at the Bangladesh Army University of Science and Technology (BAUST), Saidpur. He has a strong interest in App Development, Machine Learning, Digital Image Processing, Python, Web Development with the Django web framework, and Webflow Development. His contribution to this project focused on the Front-end Design and Testing.

Project:

- **Title:** AgroSense: An IoT-Based Smart Farming and Crop Monitoring System.

Research:

- *An Intelligent Machine Learning-Driven Handover Prediction Framework for Non-Terrestrial Networks.*
- *A Multi Engine Cognitive Framework for Ethical Decision-Making in Autonomous Vechicles:The Zihan-CED Algorithm.*

Zihan Hossain
+880 1927-699316
220201067@baust.edu.bd
Saidpur, Nilphamari, Bangladesh